XIMOD Architect — XML Builder

사용자 설명서

당신의 모드를 위해 설계된 XML 설치 프로그램.

Skyrim · Fallout · Starfield · Palworld · 외 다수

Version 2.2.2

크로스 플랫폼: Windows · Linux · macOS



목차

1. 개요 6

1.1 XIMOD Architect란 무엇인가? 6

1.2 FOMOD 형식 6

1.3 설치 및 시스템 요구 사항 7

Windows 7

Linux 7

macOS 7

1.4 파일 구조 7

2. 시작하기 9

2.1 애플리케이션 실행 9

2.2 인터페이스 개요 9

2.3 새 프로젝트 만들기 10

2.4 프로젝트 열기 및 저장 10

2.5 첫 실행 10

2.6 자유 창과 키보드 탐색 10

3. 메뉴 12

3.1 파일 메뉴 12

3.2 편집 메뉴 13

3.3 프로젝트 메뉴 13

3.4 도구 메뉴 14

3.5 도움말 메뉴 15

4. 기능 상세 설명 16

4.1 모드 정보 탭 16

4.2 설치 단계 탭 16

그룹 및 선택 유형 17

옵션 및 기본 유형 17

플러그인 파일 17

그룹 / 페이지 대상 18

조건 플래그 18

4.3 필수 설치 항목 탭 19
4.4 조건부 설치 항목 탭 19
4.5 "설정" 창 19
4.6 저장 전후 스크립트 20
예시 — 저장 전 스크립트 (Windows) 21
예시 — 저장 후 스크립트 (Windows) 21
Linux / macOS에서 이에 상응하는 방법 21
4.7 "정보" 창 22
4.8 FOMOD 파일 생성 22
4.9 번역 편집기 22
4.10 XML 편집기 24
4.11 글꼴 관리 25
4.12 두 개의 FOMOD 병합 26
4.13 배포 아카이브 내보내기 26
4.14 "속성" 창 26
4.15 명령줄 모드 (CLI) 28
4.16 업데이트 확인 28
5. FOMOD 태그 참조 29
5.1 info.xml 파일 29
5.2 ModuleConfig.xml 파일 29
루트 및 헤더 29
파일 및 폴더 29
단계 및 그룹 30
옵션 (플러그인) 30
종속성 및 조건 31
6. ModuleConfig.xml 파일 상세 설명 32
6.1 설치 프로그램이 파일을 읽는 방식 32
6.2 루트와 헤더 32
6.3 파일 설치 32
6.4 단계, 그룹 및 옵션 33

6.5 옵션의 유형 및 동작 34
6.6 플래그, 종속성 및 조건부 설치 34
7. ModuleConfig.xml 예제 36
7.1 최소 예제 — 필수 파일 36
7.2 배타적 선택 — SelectExactlyOne 36
7.3 조건부 설치 — 플래그 36
7.4 한 페이지, 여러 단계 — SelectAny + 플래그 37
7.5 사용자 지정 가능한 기본값을 갖춘 프리셋 39
7.6 소개 페이지와 마무리 페이지 42
8. 버전 2 — 새 기능 44
8.1 대상 충돌 검출 44
8.2 7z 내보내기 44
8.3 재사용 가능한 템플릿 44
8.4 폴더에서 새로 만들기 44
8.5 끌어다 놓기로 파일 지정 44
8.6 FOMOD 비교 44
8.7 시각적 조건 편집기 44
8.8 이미지 확인 및 최적화 45
8.9 명령줄 추가 사항 45
8.10 백그라운드 내보내기 및 검증 45
8.11 충돌 복구 45
8.12 더 명확해진 인터페이스 45
8.13 기존 FOMOD 번역하기 45
8.14 모드 아카이브 열기 46
8.15 가져오기 충실도 47
8.16 백업 및 복구 47
8.17 자동 조건으로서의 마스터 47
8.18 Light 플러그인(ESL)과 플러그인 보고서 48
8.19 더 풍부한 시뮬레이터와 시나리오 48
8.20 옵션별 설치 크기 48

8.21 Nexus 설명 생성기 48
8.22 편집 가능한 조건 편집기 49
8.23 프로젝트 문자열 표 49
8.24 BSA 및 BA2 아카이브 읽기 49
8.25 중첩 조건 그룹 및 버전 조건 50
8.26 게임 및 해당 카테고리 편집 51
8.27 트리 색상 51
8.28 마지막 위치를 기억하는 파일 대화 상자 51
8.29 프리셋 마법사 51
8.30 FOMOD 간 잘라내기, 복사, 붙여넣기 52
8.31 디스크에서 소스 이름 바꾸기 53
8.32 누락된 파일 찾기 53
부록 53
부록 A — Config.ini 파일 53
부록 B — 키보드 단축키 54
부록 C — 용어집 55
부록 D — 완전한 FOMOD 예제 57
D.1 info.xml 57
D.2 ModuleConfig.xml 57
부록 E — 라이선스 및 출처 표기 61

1. 개요

1.1 XIMOD Architect란 무엇인가?

XIMOD Architect는 게임 모드(Skyrim, Fallout, Starfield 및 Vortex나 Mod Organizer 2가 관리할 수 있는 기타 모든 게임)용 FOMOD 설치 프로그램을 생성하기 위한 크로스 플랫폼 도구입니다. 이 도구를 사용하면 단계, 옵션 그룹, 옵션, 조건부 파일로 구성된 완전한 설치 마법사를 시각적으로 구축한 다음, 모드 관리자가 해석할 수 있는 표준화된 XML 파일을 자동으로 생성할 수 있습니다.

FOMOD는 게임 형식이 아니라 모드 관리자 형식입니다. 직접 만든 설치 프로그램은 모든 베데스다 게임은 물론, Vortex나 Mod Organizer 2가 관리할 수 있는 대부분의 다른 게임에서도 작동합니다 — Vortex로 모드를 적용할 수 있는 게임이라면 FOMOD 설치 프로그램을 사용할 수 있습니다. Categories.json 파일의 기본 제공 목록(베데스다 타이틀 11개: Skyrim의 세 가지 에디션 — Legendary, Special Edition, VR —, Fallout 3, Fallout: New Vegas, Fallout 4 및 Fallout 4 VR, Oblivion과 Oblivion Remastered, Morrowind, 그리고 Starfield)은 info.xml의 게임과 Nexus 카테고리를 채우는 데에만 쓰이며, 설치 프로그램을 사용할 수 있는 곳을 제한하지 않습니다. 이 목록은 외부 파일이며 편집이 가능합니다. 도구 → 게임 / 카테고리…(§ 8.26)로 애플리케이션을 재컴파일하지 않고도 다른 게임을 얼마든지 추가할 수 있습니다.

이 소프트웨어는 기존 FOMOD Creation Tool을 Rust 언어로 재작성한 것입니다. 이러한 전면 개편을 통해 성능이 향상되었으며, Windows, Linux, macOS에서 동일한 동작을 보장하고 현대적인 인터페이스를 제공합니다. 또한 XIMOD Architect는 매우 진보된 국제화 시스템으로 두각을 나타냅니다. 인터페이스를 다양한 언어로 번역할 수 있으며, 각 문자 체계에 적합한 글꼴을 동적으로 불러옵니다.

주요 기능은 다음과 같습니다:

- 완전한 FOMOD 설치 패키지 생성;
- 다단계 설치 마법사;
- 다양한 선택 유형(정확히 하나, 최소 하나, 아무 개나 등)을 지원하는 옵션 그룹;
- 사용자의 선택에 따른 조건부 파일 설치;
- 옵션의 유형을 결정하는 종속성 패턴;
- 설치 흐름을 제어하는 "플래그"(조건 플래그) 시스템;
- 저장 전후에 실행되는 사용자 정의 스크립트;
- 다국어 인터페이스 및 동적 글꼴;
- 네이티브 방식의 투명한 스플래시 화면.

1.2 FOMOD 형식

FOMOD("Fallout Mod")는 베데스다 게임용 모드 설치 프로그램의 표준 형식이며, Vortex와 Mod Organizer 2 같은 모드 관리자는 대부분의 다른 게임에도 이 형식을 사용합니다. FOMOD 설치 프로그램은 fomod라는 하위 폴더에 모여 있는 두 개의 XML 파일을 기반으로 합니다:

- info.xml — 모드 메타데이터: 이름, 제작자, 버전, 웹사이트, 설명, 카테고리.
- ModuleConfig.xml — 설치 구성: 단계, 그룹, 옵션, 파일, 조건.

XIMOD Architect에서 생성된 파일은 주요 모드 관리자와 호환됩니다:

- Mod Organizer 2 (MO2);

- Vortex;
- Nexus Mod Manager (NMM).

> 참고 — fomod 폴더의 이름과 info.xml, ModuleConfig.xml이라는 이름은 사양에 의해 정해져 있으며, 모드 관리자가 그대로 인식합니다. 이 이름들은 절대로 변경해서는 안 됩니다.

1.3 설치 및 시스템 요구 사항

시스템 요구 사항(참고용):

항목	권장 사양
시스템	Windows 10 이상, Linux(최신 glibc) 또는 macOS 11 이상.
디스플레이	OpenGL 3.3과 호환되는 그래픽 카드(egui의 "glow" 렌더러).
RAM	수백 MB — 이 애플리케이션은 용량이 가볍습니다.
디스크 공간	수십 MB로, Noto 글꼴과 assets/에 제공된 플래그에 따라 다릅니다.

Windows

실행 파일 ximod-architect.exe, assets 폴더(data, locales, fonts, images, icons), 그리고 선택 사항으로 스플래시 화면용 파일 assets/images/splash.png를 같은 폴더에 배치하십시오. 애플리케이션 아이콘은 컴파일 시 실행 파일에 내장됩니다. 그런 다음 ximod-architect.exe를 실행하십시오.

Linux

설치 스크립트는 packaging/linux에 제공됩니다:

```
# System-wide installation (requires sudo)
sudo ./packaging/linux/install.sh

# Installation for the current user
./packaging/linux/install.sh --user
```

소스에서 빌드하려면 Rust 1.99.0 이상이 필요하며, Linux에서는 다음 개발 라이브러리가 필요합니다: libgtk-3-dev, libxcb-render0-dev, libxcb-shape0-dev, libxcb-xfixes0-dev, libspeechd-dev, libxkbcommon-dev, libssl-dev, libx11-dev.

macOS

스크립트는 배포 준비가 완료된 애플리케이션 번들(.app)을 생성합니다:

```
./packaging/macos/create-bundle.sh
```

"XIMOD Architect.app" 번들이 dist 폴더에 생성됩니다. 그런 다음 hdiutil 유틸리티로 디스크 이미지(.dmg)를 만들 수 있습니다.

1.4 파일 구조

실행 파일과 함께 제공되는 파일 트리는 다음과 같습니다:

```
ximod-architect(.exe)            → executable
Config.ini                       → configuration (created on first launch)
assets/
   data/
      Categories.json            → games and Nexus categories
      Languages.json             → 6,777 languages (endonym, ISO codes, font)
      Countries.json             → 244 countries (names, flag, official languages)
      CountryLanguages.json      → languages spoken by country
   locales/                      → translations (one folder per language)
      eng/main.ftl
      fra/main.ftl
      …                          → 32 languages provided
   fonts/                        → Noto fonts (one per writing system)
   icons/                        → application icons
   images/
      splash.png                 → splash screen
      svg/                       → country flags (244 SVG files)
```

2. 시작하기

2.1 애플리케이션 실행

시작 시 XIMOD Architect는 먼저 페이드 효과가 있는 네이티브 방식의 투명한 스플래시 화면을 표시하며, 그 지속 시간은 조정할 수 있습니다(기본값 2초이며, 화면을 비활성화할 수도 있습니다). 그런 다음 기본 디스플레이의 중앙에 메인 창이 열립니다. 초기 크기는 구성 파일에서 가져오며(기본값 1280 × 800), 800 × 600보다 작아질 수 없습니다. 제목 표시줄에는 애플리케이션 이름과 그 버전 번호가 표시됩니다.

2.2 인터페이스 개요

메인 창은 다섯 개의 영역으로 구성됩니다:

- 상단의 메뉴 바 — 파일, 편집, 프로젝트, 도구, 도움말 — 와 그 바로 아래의 도구 모음으로, 일상적인 작업(새로 만들기, 열기, 저장, 실행 취소, 다시 실행, 검증, 미리 보기, 내보내기)을 담고 있습니다. chapter 3을 참조하십시오;
- 왼쪽의 프로젝트 트리: 모드 정보, 그다음으로 그룹과 옵션이 있는 설치 단계, 필수 파일, 조건부 설치. 노드를 클릭하면 편집할 수 있고, 마우스 오른쪽 버튼을 클릭하면 해당 작업(추가, 복제, 이동, 삭제, 템플릿으로 저장)을 볼 수 있으며, 옵션을 다른 그룹 위로, 그룹을 다른 단계 위로, 단계를 다른 단계 앞으로 끌어다 놓아 이동할 수 있습니다. "설치 단계"와 "조건부 설치 항목" 옆의 + 버튼은 요소를 바로 추가합니다;
- 중앙의 인스펙터는 선택한 노드의 편집기를 표시합니다: 모드 정보, 단계(이름, 표시 조건, 페이지 대상, 그룹 목록), 그룹(이름, 선택 유형, 그룹 대상, 옵션 목록), 옵션(설명, 이미지, 유형, 플래그, 종속성 패턴, 파일), 필수 파일 또는 조건부 세트. 인스펙터 상단의 경로 표시줄(breadcrumb)을 통해 한 단계 위로 올라갈 수 있습니다;
- 하단의 문제 패널은 프로젝트 → FOMOD 유효성 검사 이후에 표시됩니다. 각 발견 항목이 심각도와 함께 나열되며, 클릭하면 트리에서 해당 노드가 선택됩니다. 트리 자체에도 해당 노드 옆에 작은 오류/경고 카운터가 표시됩니다;
- 맨 아래의 상태 표시줄: 왼쪽에는 최신 메시지(시작 시: "준비 완료"); 오른쪽에는 단계와 옵션의 개수가, 그리고 프로젝트에 저장되지 않은 변경 사항이 있을 때는 점과 함께 "수정됨"이 표시됩니다. 성공, 경고, 오류는 오른쪽 상단 모서리에 알림으로도 나타납니다.

인터페이스의 외관(어두운 테마, 밝은 테마 또는 시스템 테마, 글꼴 크기, 언어)은 § 4.5에 설명된 설정 창에서 지정합니다.

트리에서 클릭하면 노드가 선택되어 인스펙터에 표시되고, 단계나 그룹을 더블 클릭하면 같은 동작과 함께 해당 항목이 열립니다. 단계의 오른쪽 클릭 메뉴는 펼치기(모두, 선택 항목, 선택 항목부터 — 해당 단계와 그 아래의 모든 항목)와 접기(동일한 세 가지 선택)를 제공하며, 그룹의 메뉴는 통상적인 추가 / 복제 / 이동 / 삭제 명령에 앞서 펼치기와 접기(모두, 선택 항목)를 제공합니다. 각 탭은 자체 문제 패널을 유지합니다. 표시되는 발견 항목은 항상 활성 탭에 로드된 FOMOD에 속하며, 한 탭에서 시작한 검증은 그 사이에 다른 탭으로 전환하더라도 해당 탭에서 완료됩니다.

트리는 어떤 텍스트 필드에도 포커스가 없을 때 키보드로도 조작할 수 있습니다. ↑와 ↓는 커서 링을 행에서 행으로 이동하고(선택한 노드에서 시작합니다), Enter는 그 아래의 행을 선택하며, →는 닫힌 단계나 그룹을 펼치고(열린 항목에서는 첫 번째 자식으로 이동), ←는 열린 항목을

접습니다(닫힌 행이나 옵션에서는 그 상위 항목으로 돌아갑니다). 링은 이동할 때 보이도록 스크롤됩니다.

2.3 새 프로젝트 만들기

설치 프로그램 생성은 일반적으로 아래 단계를 따릅니다:

- 새 프로젝트 — 파일 메뉴 → 새로 만들기, 그런 다음 모드 파일이 들어 있는 루트 디렉터리를 선택합니다(모드 정보 탭의 "찾아보기..." 버튼).
- 정보 입력 — 모드 정보 탭에서 모드 이름, 제작자, 버전, 게임, 카테고리, 웹사이트, 헤더 이미지, 설명을 입력합니다.
- 마법사 구성 — 설치 단계 탭에서 단계를 추가한 다음, 옵션 그룹을, 그다음 옵션을 추가합니다.
- 파일 연결 — 각 옵션에 대해 설치할 파일이나 폴더를 추가한 다음, 필요하면 표에서 대상을 지정합니다(기본값은 비어 있으며, 이 경우 게임의 Data 폴더 루트에 설치됩니다).
- 조건 정의 — 플래그와 조건부 설치를 사용하여 사용자의 선택에 맞게 설치를 조정합니다.
- 저장 — 파일 메뉴 → 저장 (Ctrl+S): XIMOD Architect가 fomod 폴더를 만들고 그 안에 info.xml과 ModuleConfig.xml을 기록합니다.

2.4 프로젝트 열기 및 저장

저장 시 항상 루트 디렉터리 아래에 fomod 하위 폴더가 생성된 다음, 그 안에 두 개의 XML 파일이 UTF-8로 기록됩니다. 루트 디렉터리가 정의되지 않은 한 저장 명령은 비활성화된 상태로 유지됩니다.

두 가지 열기 모드가 제공됩니다. "폴더 열기..."는 루트 디렉터리를 묻고 그 fomod 하위 폴더에서 프로젝트를 로드합니다. "파일 열기..."는 XML 파일을 직접 묻습니다. 이 경우 루트 디렉터리가 추론됩니다(해당 파일이 fomod 하위 폴더에 있다고 가정합니다). 읽을 때 XIMOD Architect는 파일 인코딩(바이트 순서 표시 유무에 관계없는 UTF-8과 UTF-16)을 자동으로 인식하므로, 다른 도구로 생성된 설치 프로그램을 다시 열 수 있습니다.

성공적으로 열린 각 프로젝트는 최근 파일 목록의 맨 위에 추가되며, 파일 → 최근 항목을 통해 접근하고 설정의 최근 파일 탭에서 관리할 수 있습니다.

2.5 첫 실행

맨 처음 실행할 때 XIMOD Architect는 영어로 시작하며, 사용자가 자신의 국가와 언어를 선택할 수 있도록 설정 창을 자동으로 엽니다.

이 동작은 Config.ini 파일의 FirstStart 키에 의해 제어됩니다. 초기 구성이 저장되지 않은 한 값은 0이며, "저장"을 처음 클릭하는 즉시 1로 전환됩니다. 이후의 실행에서는 선택한 언어로 바로 시작합니다.

사용자가 "취소"로 창을 닫으면 키는 0으로 유지되며, 다음 실행 시 창이 다시 열립니다.

2.6 자유 창과 키보드 탐색

여러 도구가 자유(독립) 창으로 열립니다: "국가 선택" 선택기, "속성" 창, 번역 편집기, XML 편집기. 각 창은 두 번째 화면을 포함하여 자유롭게 이동하고 크기를 조정할 수 있으며, 메인 창과 나란히 작동합니다.

중앙 배치 및 유지 — 처음 열릴 때 자유 창은 XIMOD 메인 창 중앙에 배치됩니다. 사용자가 창을 이동하거나 크기를 조정하면, 창을 닫을 때(그리고 애플리케이션을 종료할 때) 그 위치와 크기가 기억되었다가 다음에 열릴 때 복원됩니다. 이 값들은 Config.ini의 [WindowPositions] 섹션에 보관됩니다(부록 A 참조). 메인 창 자체는 크기는 기억하지만 위치는 기억하지 않으며, 실행할 때마다 중앙에 배치된 상태로 유지됩니다.

Esc로 닫기 — Esc 키는 활성 자유 창과 메인 창의 모달 대화 상자(설정, 정보, 확인, 검증 보고서, 스크립트)를 닫습니다. 확인 대화 상자의 경우 Esc는 "취소"로 작동하며 작업을 절대 실행하지 않습니다. 두 가지 예외: 번역 편집기(입력 중 실수로 닫히는 것을 방지하기 위해)와 XIMOD 메인 창은 Esc로 닫히지 않습니다.

키보드 탐색 — "속성" 창의 목록, 번역 편집기 표, 국가 선택기의 플래그 격자는 모두 키보드로 조작할 수 있습니다: 방향키(및 숫자 키패드), Page Up/Page Down, Home/End, Enter, Tab/Shift+Tab. 각 창에 특화된 세부 사항은 해당 섹션에 설명되어 있으며 부록 B에 요약되어 있습니다.

필터 지우기 — 모든 "필터:" 필드에는 오른쪽에 지우기 아이콘(빨간 십자)이 있어 한 번의 클릭으로 필터를 비웁니다.

3. 메뉴

메뉴 바에는 다섯 개의 메뉴가 있습니다: 파일, 편집, 프로젝트, 도구, 도움말. 그 아래의 도구 모음은 일상적인 작업(새로 만들기, 열기, 저장, 검증, 미리 보기, 내보내기)에 원클릭으로 접근할 수 있게 합니다. 표시된 단축키는 전역적으로도 활성화되어 있으며(Ctrl 키는 macOS의 Cmd에 해당합니다), 모달 대화 상자가 열려 있는 동안에만 무시됩니다. 모달 대화 상자는 창의 나머지 부분을 어둡게 하며, Enter는 기본 버튼을 실행하고 Esc는 취소합니다.

3.1 파일 메뉴

명령	단축키	설명
새로 만들기	Ctrl+N	새 빈 프로젝트를 새 탭에서 엽니다(현재 탭이 아직 손대지 않은 상태인 경우에만 대체됩니다).
폴더에서 새로 만들기…	—	모드 폴더를 스캔하여 각 하위 폴더가 하나의 옵션이 되는 뼈대를 제안합니다. § 8.4를 참조하십시오.
폴더 열기…	Ctrl+O	루트 디렉터리를 선택하고 그 fomod 하위 폴더에서 프로젝트를 로드합니다.
파일 열기…	Ctrl+Shift+O	info.xml 또는 ModuleConfig.xml을 직접 선택합니다. 루트 디렉터리는 추론됩니다. fomod 폴더 안에 있지 않은 파일은 거부됩니다.
아카이브 열기…	—	다운로드한 .zip 또는 .7z 모드 아카이브(Nexus에서 받은 것처럼)를 구성 디렉터리의 작업 폴더로 추출한 다음, 그 FOMOD를 엽니다. 이미 추출된 아카이브는 재사용됩니다. .rar 아카이브는 거부됩니다. 먼저 7-Zip으로 추출하십시오. 아카이브를 창에 끌어다 놓아도 동일하게 작동합니다. § 8.14를 참조하십시오.
저장	Ctrl+S	fomod/info.xml과 fomod/ModuleConfig.xml을 생성합니다. 프로젝트에 아직 루트 디렉터리가 없으면 먼저 폴더를 묻습니다.
다른 이름으로 저장…	Ctrl+Shift+S	프로젝트의 (새) 루트 디렉터리를 선택한 다음 그곳에 저장합니다.
백업 복원…	—	fomod/backups 아래에 보관된 FOMOD XML 파일의 이전 버전을 현재 프로젝트와의 차이 개수와 함께 나열합니다. 각 버전은 비교하거나 편집기로 복원할 수 있습니다. 프로젝트에 아직 백업이 없으면 비활성화됩니다. § 8.16을 참조하십시오.
FOMOD 병합…	—	다른 FOMOD의 단계, 필수 파일, 조건부 설치를 현재 프로젝트에 추가합니다. 루트 디렉터리가 정의되지 않은 한 비활성화됩니다. § 4.12를 참조하십시오.

명령	단축키	설명
배포 아카이브 내보내기...	—	FOMOD를 기록한 다음 전체 루트 디렉터리를 게시 준비가 된 .zip 또는 .7z 아카이브로 백그라운드에서 진행 창과 함께 압축합니다. § 4.13 및 § 8.10을 참조하십시오.
FOMOD 닫기	—	활성 탭을 닫으며, 저장되지 않은 변경 사항이 있으면 먼저 묻습니다.
모든 FOMOD 닫기	—	모든 탭을 닫으며, 저장되지 않은 변경 사항이 있는 탭이 있으면 먼저 묻습니다.
최근 항목	—	가장 최근에 열었던 프로젝트를 나열하는 하위 메뉴입니다. 목록이 비어 있으면 "(비어 있음)"을 표시합니다.
종료	Ctrl+Q	애플리케이션을 닫습니다. 저장되지 않은 프로젝트가 나열되며 먼저 저장할 수 있습니다. 종료 시 구성이 저장됩니다.

3.2 편집 메뉴

명령	단축키	설명
실행 취소	Ctrl+Z	프로젝트에 대한 마지막 변경을 되돌립니다(같은 필드에서의 키 입력은 병합됩니다). 되돌릴 것이 없으면 비활성화됩니다.
다시 실행	Ctrl+Y	마지막으로 되돌린 변경을 다시 적용합니다(Ctrl+Shift+Z도 작동합니다).
잘라내기	Ctrl+X	트리에서 선택한 단계, 그룹 또는 옵션을 제거하고 클립보드에 보관합니다. 편집 → 실행 취소로 원래대로 되돌릴 수 있습니다. § 8.30을 참조하십시오.
복사	Ctrl+C	선택한 단계, 그룹 또는 옵션을 그 안에 포함된 모든 것과 함께 클립보드에 — 그리고 시스템 클립보드에도 — 복사합니다. § 8.30을 참조하십시오.
붙여넣기	Ctrl+V	클립보드의 내용을 선택 항목 옆에 삽입합니다: 단계는 선택한 단계 뒤에, 그룹은 선택한 단계 안에, 옵션은 선택한 그룹 안에 들어갑니다. § 8.30을 참조하십시오.

3.3 프로젝트 메뉴

편집 중인 FOMOD에 작용하는 모든 것.

명령	단축키	설명

명령	단축키	설명
FOMOD 유효성 검사	—	프로젝트와 ModuleConfig.xml이 ModConfig 5.0 스키마에 부합하는지 확인하고, 참조된 파일, 대상 충돌, 이미지를 (백그라운드에서) 검증한 다음, 상세 보고서를 표시합니다. § 4.8을 참조하십시오.
설치 프로그램 미리 보기…	—	모드 관리자가 보여 주는 것처럼 설치 프로그램을 시뮬레이션합니다. § 4.11을 참조하십시오.
누락된 파일 찾기…	—	프로젝트가 사용하지만 더 이상 존재하지 않는 파일, 폴더, 이미지를 나열하고 어디로 옮겨졌는지 제안합니다. § 8.32를 참조하십시오.
조건 편집기…	—	모든 조건 플래그를 그것이 설정되고 사용되는 위치와 함께 보여 주며(클릭 가능), 플래그를 모든 곳에서 이름 변경하거나 제거하고, 선택한 노드에 대한 조건 빌더를 제공합니다. § 8.7 및 § 8.22를 참조하십시오.
FOMOD 번역…	—	기존 설치 프로그램(활성 프로젝트, 폴더 또는 아카이브)을 그 XML 구조를 변경하지 않고 번역하는 창을 엽니다. § 8.13을 참조하십시오.
비교…	—	프로젝트를 다른 FOMOD와 비교하여 변경된 사항을 나열합니다. § 8.6을 참조하십시오.
템플릿…	—	선택한 단계를 재사용 가능한 템플릿으로 저장하거나, 저장된 템플릿을 삽입합니다. § 8.3을 참조하십시오.
프리셋 마법사…	—	현재 프로젝트에 "프리셋 + 사용자 지정" 설치 프로그램을 생성합니다. 프리셋이 있는 설치 모드 페이지, 필요할 때 표시되는 기능 페이지, 동적 기본값, 조건부 설치로 구성됩니다. § 7.5 및 § 8.29를 참조하십시오.
XML 편집기	—	info.xml과 ModuleConfig.xml에 접근할 수 있는 하위 메뉴입니다. § 4.10을 참조하십시오.

3.4 도구 메뉴

특정 프로젝트에 매이지 않은 설정 및 유틸리티.

명령	단축키	설명
설정…	Ctrl+,	설정 창(국가, 언어, 테마, 글꼴 등)을 엽니다. § 4.5를 참조하십시오.
트리 색상…	—	프로젝트 트리의 행 종류(단계, 그룹, 옵션…)마다 테마별로 색상을 하나씩 선택합니다. § 8.27을 참조하십시오.
저장 전 스크립트…	—	저장 전에 실행되는 스크립트를 편집합니다. § 4.6을 참조하십시오.

명령	단축키	설명
저장 후 스크립트…	—	저장 후에 실행되는 스크립트를 편집합니다. § 4.6을 참조하십시오.
플러그인 보고서…	—	프로젝트가 설치하는 모든 플러그인 파일(.esp, .esm, .esl)의 표입니다: 종류, light 플래그, 마스터(누락된 것은 강조 표시), light 플러그인 한도 대비 새 레코드, ESL 적격성. § 8.18을 참조하십시오.
아카이브 내용…	—	베데스다 .bsa 또는 .ba2 아카이브에 담긴 파일을 검색 필드와 함께 나열합니다(형식, 크기, 압축). § 8.24를 참조하십시오.
Nexus 설명…	—	프로젝트로부터 Nexus Mods 페이지 텍스트(BBCode 또는 Markdown)를 생성합니다: 요구 사항, 설치 옵션, 설치 참고, 이전 버전 대비 변경 로그. 복사하거나 저장할 수 있습니다. § 8.21을 참조하십시오.
프로젝트 문자열…	—	프로젝트의 모든 이름과 설명을 검색, 찾기/바꾸기, 중복 검출과 함께 보여 주는 표입니다. 텍스트는 그 자리에서 편집됩니다. § 8.23을 참조하십시오.
인터페이스 번역…	—	인터페이스 번역 편집기를 엽니다. § 4.9를 참조하십시오.
속성…	—	국가 / 언어 데이터베이스의 읽기 전용 탐색기를 엽니다. § 4.14를 참조하십시오.
게임 / 카테고리…	—	게임 편집기를 엽니다. Categories.json을 직접 편집하지 않고도 게임과 해당 Nexus 카테고리(이름, slug, 기본 마스터, 경량 플러그인 제한)를 추가, 수정 또는 제거할 수 있습니다. § 8.26을 참조하십시오.

3.5 도움말 메뉴

명령	단축키	설명
사용자 설명서	F1	애플리케이션과 함께 제공되는 Manuals 폴더에서 인터페이스 언어로 된 이 설명서(PDF)를 엽니다.
업데이트 확인…	—	최신 게시 버전을 조회하고 다운로드 링크를 제공합니다.
정보	—	"정보" 창(이름, 버전, 라이선스)을 표시합니다. § 4.7을 참조하십시오.

4. 기능 상세 설명

4.1 모드 정보 탭

이 탭은 모드 메타데이터와 루트 디렉터리 선택을 한데 모아 둡니다.

작업 공간 — "루트 디렉터리:" 줄은 작업 폴더(정의되지 않은 경우 "(디렉터리 선택)")를 표시합니다. "찾아보기…" 버튼으로 이를 선택할 수 있습니다.

메타데이터 필드는 다음과 같습니다:

필드	역할
모드 이름	설치 프로그램의 표시 이름(필수).
제작자	모드 제작자의 이름.
버전	버전 번호(새 프로젝트의 경우 기본값 1.0.0).
게임 이름	Categories.json에서 가져온 목록에서 선택하는 대상 게임.
카테고리	모드의 카테고리. 선택한 게임에 따라 달라지며, 그렇지 않으면 기본 목록이 제공됩니다(Animation, Armour, Audio, Textures, Weapons 등).
웹사이트	모드 페이지의 주소.
헤더 이미지	설치 프로그램 상단에 표시되는 이미지. "찾아보기…" 및 "지우기" 버튼이 있으며 미리 보기가 표시됩니다. 경로는 루트 디렉터리를 기준으로 상대 경로로 저장됩니다.
설명	모드의 소개 텍스트(여러 줄 영역).

> 참고 — 필드를 변경하면 상태 표시줄에서 프로젝트가 "수정됨"으로 표시됩니다.

Nexus 링크 — "게임 이름" 선택기 옆의 "Nexus ↗" 버튼은 선택한 게임의 Nexus Mods 페이지(그 "slug"로 구성됨)를 브라우저에서 바로 엽니다. 선택한 게임에 slug가 있는 경우에만 활성화됩니다.

"게임 이름"과 "카테고리" 목록은 외부 파일 assets/data/Categories.json에서 가져오며, 재컴파일 없이 편집할 수 있습니다. 각 게임은 그 파일에 식별자, 이름, Nexus "slug", 그리고 카테고리 목록으로 기술됩니다:

```
{
  "games": {
    "skyrimSpecialEdition": {
      "name": "The Elder Scrolls V: Skyrim Special Edition",
      "nexusSlug": "skyrimspecialedition",
      "categories": ["Animation", "Armour", "Audio", "…"]
    }
  }
}
```

4.2 설치 단계 탭

이것은 설치 마법사의 핵심입니다. 단계 → 그룹 → 옵션이라는 3단계 계층 구조를 기반으로 하며, 이는 왼쪽 프로젝트 트리의 형태와 정확히 일치합니다. "설치 단계" 옆의 + 버튼(기본값 "단계 N"으로 이름 지어짐)이나 빈 인스펙터에서 단계를 추가합니다. 트리에서 단계, 그룹 또는 옵션을 선택하여 중앙에서 편집합니다. 단계 편집기는 이름(편집 가능), "단계 삭제" 버튼(확인을 묻는 유일한 삭제입니다. 그 외의 모든 것은 Ctrl+Z로 되돌릴 수 있습니다), 표시 조건, 페이지 대상,

그리고 "그룹 추가" 버튼이 있는 그룹 목록을 보여 줍니다. 그룹 편집기는 이름, 선택 유형, 그룹 대상, 그리고 "옵션 추가" 버튼이 있는 옵션 목록을 보여 줍니다. 옵션 편집기는 아래에 설명되어 있습니다.

그룹 및 선택 유형

각 그룹에는 이름과, 최종 사용자가 몇 개의 옵션을 선택할 수 있는지를 결정하는 선택 유형이 있습니다. 사용 가능한 다섯 가지 유형은 다음과 같습니다:

유형	의미
정확히 하나(필수) — SelectExactlyOne	사용자는 정확히 하나의 옵션을 선택해야 합니다.
최대 하나 — SelectAtMostOne	최대 하나의 옵션(또는 선택 안 함).
개수 제한 없음 — SelectAny	임의의 개수의 옵션(기본값).
모두(선택 불가) — SelectAll	모든 옵션이 선택됩니다.
최소 하나 — SelectAtLeastOne	최소 하나의 옵션.

단계 편집기의 "그룹 추가" 버튼(또는 단계의 오른쪽 클릭 메뉴)은 기본적으로 "개수 제한 없음" 유형의 그룹을 생성합니다. 그룹의 오른쪽 클릭 메뉴나 그룹 편집기의 "삭제"는 그룹을 제거합니다.

순서 변경 — 모든 것을 다시 만들지 않고 표시 순서를 조정합니다: 트리에서 노드를 끌어 가야 할 곳에 놓거나(옵션을 다른 옵션 앞으로 또는 그룹 위로, 그룹을 다른 그룹 앞으로 또는 단계 위로, 단계를 다른 단계 앞으로), 노드의 오른쪽 클릭 메뉴에서 "앞으로 이동" / "뒤로 이동"을 사용합니다. "복제"는 단계, 그룹 또는 옵션을 그 안에 포함된 모든 것과 함께 복사합니다. 모든 변경은 편집 → 실행 취소 (Ctrl+Z)로 되돌리고 편집 → 다시 실행 (Ctrl+Y)로 다시 적용할 수 있습니다. 한 필드에서의 연속된 키 입력은 하나의 단계로 간주됩니다. 같은 오른쪽 클릭 메뉴나 편집 메뉴, 또는 Ctrl+X / Ctrl+C / Ctrl+V로 사용할 수 있는 잘라내기, 복사, 붙여넣기는 노드를 프로젝트의 다른 위치나 열려 있는 다른 FOMOD로 옮깁니다(§ 8.30).

옵션 및 기본 유형

옵션(FOMOD 형식에서 <plugin> 요소)은 사용자에게 제공되는 선택지입니다. 이름, 설명, 이미지(미리 보기, "찾아보기..." 및 "지우기" 버튼), 그리고 기본 유형을 갖습니다:

유형	의미
선택 — Optional	선택적 옵션(기본값).
필수 — Required	항상 설치되는 옵션.
권장 — Recommended	권장되지만 선택적인 옵션.
사용 불가 — NotUsable	사용할 수 없는 옵션.
사용 가능할 수 있음 — CouldBeUsable	조건에 따라 사용될 수 있는 잠재적 옵션.

플러그인 파일

"소스" 섹션은 옵션이 설치하는 파일과 폴더를 네 개의 열로 된 표(유형, 소스, 대상, 우선순위)로 제시합니다. "파일 추가", "폴더 추가", "제거" 버튼이 목록을 관리합니다. 대상 열은 편집 가능합니다: 셀을 클릭하고 항목이 설치되어야 하는 경로를 게임의 Data 폴더 기준 상대 경로로

입력합니다. 기본값은 비어 있으며, 이 경우 파일 — 또는 폴더의 내용 — 이 Data 루트에 설치됩니다. 이것은 소스 폴더가 이미 게임의 구조를 재현하고 있을 때(예를 들어 자체적으로 meshes, textures, SKSE 등을 포함하는 폴더) 올바른 선택입니다. 대상이 다를 때만, 예를 들어 SKSE\Plugins\MyMod.ini처럼 명시적인 경로를 입력하십시오. 파일은 루트 디렉터리 아래에 있어야 합니다. 옵션이 선택된 상태에서 파일 탐색기(Windows 탐색기, Finder…)의 파일이나 폴더를 XIMOD 창으로 바로 끌어다 놓을 수도 있습니다. 그러면 즉시 해당 옵션의 소스 표에 추가됩니다(§ 8.5 참조). 대화 상자는 이전에 선택한 폴더에서 열리므로, 같은 수준의 폴더를 클릭 한 번으로 고를 수 있습니다(§ 8.28). “디스크에서 이름 바꾸기…”는 선택한 파일이나 폴더의 이름을 모드 폴더에서 바꾸고 그것에 대한 모든 참조를 업데이트합니다(§ 8.31).

그룹 / 페이지 대상

각 파일을 일일이 편집하지 않고 여러 옵션을 같은 위치로 보내려면 일괄 대상 컨트롤을 사용하십시오. 그룹의 편집기에서 "전체 그룹 대상 위치" 필드와 그 "이 그룹의 모든 옵션에 적용" 버튼은 해당 그룹의 모든 옵션의 모든 파일에 하나의 설치 대상을 한 번에 설정합니다. 단계 수준에서는 접을 수 있는 "설치 대상 위치 (전체 페이지)" 패널이 해당 페이지의 모든 그룹에 걸쳐 모든 옵션에 대해 동일한 작업을 수행합니다. 상태 메시지가 업데이트된 파일 수를 확인해 줍니다. 옵션에 실제로 파일이 포함될 때까지 버튼은 비활성화된 상태로 유지되며, 이후에도 옵션의 소스 표에서 개별 대상을 세밀하게 조정할 수 있습니다. 이 컨트롤은 프리셋 마법사(§ 8.29)가 조건부 설치로 옮긴 파일에도 적용됩니다. 그룹 또는 페이지의 옵션에서 비롯된 조건부 세트도 함께 업데이트됩니다.

조건 플래그

"플래그 이름:" 섹션에서는 플래그를 옵션에 연결할 수 있습니다(이름 = 값 쌍). 최종 사용자가 옵션을 선택하면 그 플래그가 설정됩니다. 그런 다음 이 플래그는 단계의 표시 여부, 옵션의 유형(종속성 패턴) 또는 조건부 파일의 설치를 조건부로 제어할 수 있습니다. "플래그 추가" 및 "플래그 제거" 버튼이 목록을 관리합니다.

자동 완성 — 플래그와 종속성의 이름 및 값 필드는 입력하는 동안 프로젝트에 이미 존재하는 요소에서 가져온 제안 메뉴를 제공하여, 매칭을 조용히 깨뜨릴 수 있는 오타를 방지합니다. 파일 종속성의 경우 값은 Active / Inactive / Missing 상태를 제안합니다. 제안은 구속력이 없습니다: 새 값을 자유롭게 입력할 수 있습니다.

옵션 종속성. 플래그 아래에 있는 이 섹션은 옵션의 유형을 동적으로 만듭니다. 위의 기본 유형은 어떤 패턴도 일치하지 않을 때 적용됩니다. 각 패턴은 그 조건이 충족될 때 옵션에 다른 유형을 부여합니다. 새 패턴 / 패턴 삭제를 사용하여 목록을 관리하고, 선택한 패턴에 대해 패턴 연산자(And / Or)와 적용할 패턴 유형(Optional, Required, Recommended, NotUsable 또는 CouldBeUsable)을 선택한 다음, 조건을 추가하십시오. 조건에는 유형(File 또는 Flag), File/Flag 이름(자동 완성 포함), 그리고 파일의 경우 상태(Active, Inactive 또는 Missing) 또는 플래그의 경우 기대 값이 있습니다. 예를 들어 기본 유형을 NotUsable로 설정하고 Active 상태의 파일 DLCCoast.esm을 조건으로 하는 Optional 유형의 패턴을 추가하면, Far Harbor가 설치되어 있을 때만 옵션을 선택할 수 있게 됩니다.

단계 표시 조건. 각 단계는 조건부로 표시될 수 있습니다. 단계 이름 아래의 접을 수 있는 표시 조건 섹션은 단계의 표시 규칙을 담고 있습니다: 연산자(And / Or)를 설정하고 같은 방식으로 File 또는 Flag 조건을 추가합니다. 조건이 하나 이상 포함되면, 그 조건이 충족될 때만 단계가 마법사에

나타납니다 — 예를 들어 DLCCoast.esm이 Active일 때만 표시되는 패치 단계. 항상 단계를 표시하려면 비워 두십시오.

4.3 필수 설치 항목 탭

이 탭은 사용자의 선택과 관계없이 체계적으로 설치되는 파일과 폴더를 나열합니다. 옵션의 파일 섹션과 동일한 네 개의 열로 된 표(유형, 소스, 대상, 우선순위) — 동일하게 편집 가능한 대상 열과 함께 — 와 동일한 "파일 추가", "폴더 추가", "제거" 버튼을 제공합니다. 표 위에 있는 "대상" 필드와 그 "모든 필수 파일에 적용" 버튼은 모든 필수 파일에 하나의 설치 대상을 한 번에 설정합니다 — § 4.2의 그룹 / 페이지 대상 컨트롤에 대응하는 기능이며, 이후에도 각 대상을 세밀하게 조정할 수 있습니다.

4.4 조건부 설치 항목 탭

이 탭은 패턴을 묶습니다: 각 패턴은 조건이 충족될 때만 파일 세트를 설치합니다. "패턴 N" 탭의 행에서 패턴을 추가("➕")하거나 제거("➖")할 수 있습니다.

선택한 패턴의 경우:

- 연산자 — And 또는 Or: 패턴의 종속성이 결합되는 방식.
- 종속성 — Flag 유형([Flag] 이름 = 값) 또는 File 유형([File] 이름 (상태))의 조건 목록. "종속성 추가" 및 "종속성 제거" 버튼이 목록을 관리합니다.
- 파일 — 조건이 충족될 때 설치되는 파일의 네 개 열 표. 그 위에 있는 "대상" 필드와 그 "이 세트의 모든 파일에 적용" 버튼은 패턴의 모든 파일에 하나의 설치 대상을 한 번에 설정합니다 — 프리셋 마법사(§ 8.29)가 옵션의 파일을 이곳으로 옮긴 뒤에 유용합니다. 탭 상단의 “대상” 필드와 그 “모든 조건부 세트에 적용” 버튼은 모든 세트의 모든 파일에 하나의 설치 대상을 한 번에 설정합니다 — 프리셋 마법사를 사용한 뒤에는 옵션마다 세트가 하나씩 있을 수 있습니다.

참고 — "플러그인 종속성". 플러그인 종속성이란 단순히 파일이 게임 플러그인(.esm/.esp)인 File 종속성입니다. 이름 필드에는 모드의 표시 이름이 아니라 게임에 나타나는 그대로의 플러그인 파일명(예: Fallout4.esm, DLCCoast.esm 또는 다른 모드의 .esp)을 입력하십시오. 상태를 선택합니다: Active(플러그인이 존재하고 활성화됨 — 일반적인 선택), Inactive(존재하지만 비활성화됨) 또는 Missing(없음). File 종속성은 세 곳에서 사용할 수 있습니다: 파일을 자동으로 설치하기 위해(조건부 설치 항목 탭), 마법사 단계를 표시하거나 숨기기 위해(단계의 표시 조건), 또는 옵션의 상태를 변경하기 위해 — 예를 들어 DLC가 활성화되어 있을 때 Recommended로, 필수 플러그인이 없을 때 NotUsable로.

4.5 "설정" 창

도구 → 설정... (Ctrl+,)을 통해 접근할 수 있습니다. 일반과 최근 파일 두 개의 탭이 있습니다. 설정은 Config.ini에 보관됩니다.

일반 탭은 두 개의 열로 구성됩니다.

왼쪽 열 — 국가 플래그와 그 이름:

항목	역할
플래그	이미지를 클릭하면 국가 선택기가 열립니다(아래 참조).

항목	역할
국가 이름	선택한 언어로 쓰인 국가 이름(엔도님). 편집할 수 없는 필드이며 두 줄로 표시됩니다.

오른쪽 열 — 인터페이스 설정:

설정	설명
언어	즉시 적용되는 인터페이스 언어. 국가가 선택되지 않은 한 필드는 비활성화된 상태로 유지되며, 선택되면 해당 국가에서 사용되는 언어를 제공합니다.
테마	어둡게, 밝게 또는 시스템 설정 따름.
글꼴 크기	인터페이스 텍스트의 크기.
설명 내의 줄바꿈 처리	설명에서 \n 시퀀스를 해석합니다.
최근 파일 최대 개수	유지되는 항목의 수.
창 너비 / 높이	저장된 메인 창의 크기.

국가 선택기 — 플래그를 클릭하면 "국가 선택" 창이 열리며, 244개의 플래그를 격자로 제시하고 세로 및 가로 스크롤을 제공합니다. 이것은 자유 창으로, 열릴 때 메인 창 중앙에 배치됩니다(§ 2.6 참조). "필터:" 필드로 프랑스어 이름, 영어 이름 또는 국가 코드로 목록을 좁힐 수 있으며, 필드 오른쪽의 지우기 아이콘(빨간 십자)이 한 번의 클릭으로 필터를 비웁니다. 국가 이름은 각 플래그 아래에 나타납니다.

격자는 키보드로도 탐색할 수 있습니다: 방향키(또는 숫자 키패드 8/2/4/6)는 선택 틀을 이동하고, Page Up/Page Down은 한 화면씩 스크롤하며, Home/End는 첫 번째와 마지막 플래그로 이동하고, Tab과 Shift+Tab은 플래그를 하나씩 앞뒤로 이동하며, Enter는 강조된 국가를 확정하고 Esc는 창을 닫습니다. 세부 사항은 부록 B에 있습니다.

국가 선택은 제공되는 언어 목록을 결정합니다: XIMOD는 각 국가에서 사용되는 모든 언어를 나열하는 CountryLanguages.json 파일에 의존하며 — Papua New Guinea의 경우 최대 844개에 이르러, 드롭다운 목록이 스크롤됩니다.

버전 2에서 세 가지 설정이 추가되었습니다: 보관할 백업 수(각 저장 시 fomod/backups 아래에 보관되는 FOMOD XML의 이전 버전 수, 0 = 없음, 기본값 10); 복구 사본 자동 저장 간격 (분)(이 간격으로 수정된 모든 프로젝트의 복구본이 구성 폴더에 기록되며, 비정상 종료 후 다음 시작 시에만 제공됩니다, 0 = 끔, 기본값 5); 그리고 플러그인의 마스터를 조건으로 추가(§ 8.17 참조, 기본적으로 켜짐).

4.6 저장 전후 스크립트

XIMOD Architect는 저장 시 스크립트를 자동으로 실행할 수 있습니다: FOMOD 파일이 기록되기 바로 전에 실행되는 저장 전 스크립트와, 바로 후에 실행되는 저장 후 스크립트입니다. 이들은 모드 생성을 둘러싼 작업을 자동화하는 데 쓰입니다 — 예를 들어 저장 전 오래된 아카이브를 정리하거나, 빌드 번호를 증가시키거나, FOMOD가 기록되면 모드를 게시 준비가 된 아카이브로 자동 패키징하는 것.

이들은 도구 → 저장 전 스크립트…와 도구 → 저장 후 스크립트…를 통해 편집합니다. 창에는 도움말 텍스트, 사용 가능한 매크로 목록, 고정폭 글꼴로 된 입력 영역, 그리고 "저장"과 "취소"

버튼이 표시됩니다. 두 스크립트의 내용은 Config.ini(PreSaveScript 및 PostSaveScript 키)에 보관되므로, 한 세션에서 다음 세션으로 기억됩니다.

작동 방식: 저장 시 XIMOD는 먼저 매크로를 그 값으로 바꾸고, 결과를 임시 파일에 기록한 다음, 시스템 인터프리터로 실행합니다 — Windows에서는 cmd가 실행하는 .bat 명령 파일, Linux와 macOS에서는 sh가 실행하는 .sh 셸 스크립트. 임시 파일은 이후 삭제됩니다. 저장 전 스크립트는 XML 파일이 기록되기 전에, 저장 후 스크립트는 바로 후에 실행됩니다. 비워 둔 스크립트는 그냥 무시됩니다.

실행 전에 스크립트 텍스트에서 다음 매크로가 치환됩니다:

매크로	치환되는 값
$MODNAME$	모드 이름.
$MODAUTHOR$	제작자 이름.
$MODVERSION$	모드 버전.
$MODROOT$	루트 디렉터리 경로.
$DATE$	현재 날짜(YYYY-MM-DD).
$TIME$	현재 시각(HH:MM:SS).
$RANDOM$	난수.

예시 — 저장 전 스크립트 (Windows)

저장 전 깨끗한 폴더에서 시작하기 위해, 모드 루트에 이미 존재하는 .rar 아카이브를 삭제합니다:

```
del "$MODROOT$\*.rar" /q
```

예시 — 저장 후 스크립트 (Windows)

FOMOD가 기록되면, WinRAR을 사용하여 전체 모드 내용을 모드 이름과 버전으로 명명된 .rar 아카이브로 자동 패키징합니다:

```
d:
cd "$MODROOT$"
"C:\Program Files\WinRAR\rar.exe" a -r -ep1 "$MODNAME$_$MODVERSION$.rar" *
```

줄별로: d:는 D: 드라이브로 전환하고, cd "$MODROOT$"는 모드 루트로 이동하며, 그런 다음 WinRAR이 예를 들어 Dawnlight_1.2.0.rar로 명명된 아카이브를 생성합니다(-r 재귀, -ep1 상위 트리를 재현하지 않도록).

Linux / macOS에서 이에 상응하는 방법

Linux와 macOS에서 스크립트는 셸 스크립트로 해석됩니다. 동일한 저장 후 패키징을 예를 들어 zip으로 작성하면 다음과 같습니다:

```
cd "$MODROOT$"
zip -r "$MODNAME$_$MODVERSION$.zip" .
```

> 참고 — 이 스크립트는 사용자 권한으로 실제 시스템 명령을 실행합니다. del이나 rm 같은 명령은 파괴적입니다. 실제 모드에 사용하기 전에 버려도 되는 프로젝트에서 테스트하십시오.

4.7 "정보" 창

도움말 → 정보를 통해 열리는 이 창은 애플리케이션 이름, 버전 번호, 간략한 설명("Vortex 또는 Mod Organizer 2가 관리할 수 있는 모든 게임의 모드용 크로스 플랫폼 FOMOD 설치 프로그램 생성 도구입니다."), 그리고 "MIT 라이선스 하에 배포됩니다"라는 문구를 알려 줍니다. "확인" 버튼으로 창을 닫습니다.

4.8 FOMOD 파일 생성

저장 시 XIMOD Architect는 프로젝트를 두 개의 표준화된 XML 파일로 변환하여, fomod 하위 폴더에 UTF-8로 기록합니다.

기록하기 전에 프로젝트를 확인합니다: 이상(모드 이름 누락, 단계나 필수 파일 없음, 이름 없는 단계나 그룹, 옵션 없는 그룹)이 있으면, 창에 모두 나열되고 "그래도 저장하시겠습니까?"가 제공됩니다 — 저장은 결코 차단되지 않으므로, 작업 중인 프로젝트를 의도적으로 저장할 수 있습니다.

스키마 검증 — 이 프로젝트 확인에 더하여, 생성된 ModuleConfig.xml을 ModConfig 5.0 스키마에 대해 구조적으로 확인하는 과정이 있으며, 이는 Rust로 작성되었습니다(외부 XSD 엔진 없이): 요소 계층과 카디널리티, 필수 속성과 열거 값(선택 유형과 플러그인 유형, 파일 상태, 연산자). 각 이상은 그 줄과 열로 위치가 표시됩니다. 세 가지 방법으로 접근할 수 있습니다: 전체 보고서를 보여 주는 프로젝트 → FOMOD 유효성 검사 명령; 문서가 올바른 형식이 되는 즉시 XML 편집기에서 실시간으로; 그리고 저장 확인 창에 통합되어.

info.xml은 <fomod> 루트 아래에 메타데이터를 담고 있습니다: 이름(Name), 제작자(Author), 버전(Version), 웹사이트(Website), 설명(Description), 카테고리(Groups), 게임(Game). Game 요소는 XIMOD 고유의 확장으로, 모드 관리자가 무해하게 무시하며, 다시 로드할 때 올바른 카테고리 목록을 복원하는 데 쓰입니다.

ModuleConfig.xml은 <config> 루트 아래에 설치를 기술합니다: 모듈 이름, 헤더 이미지, 필수 파일(requiredInstallFiles), 단계(installSteps → group → plugin), 그리고 조건부 설치(conditionalFileInstalls). 각 plugin은 그 설명, 이미지, 조건 플래그, 파일, 유형을 기술합니다. 종속성은 And 또는 Or 연산자로 결합됩니다.

4.9 번역 편집기

도구 → 인터페이스 번역...을 통해 접근할 수 있는 이 도구를 사용하면 파일을 직접 편집하지 않고 인터페이스를 번역하고, 기존 번역을 수정할 수 있습니다.

창 헤더

필드	역할
플래그	국가 선택으로, 번역 가능한 언어 목록을 결정합니다.
국가 명칭	번역된 언어로 된 국가 이름으로, 선택한 글꼴로 표시됩니다.
언어 명칭	그 언어 자체로 된 언어 이름(예: "Français").
작성자	번역자의 이름.
표시된 언어	열 2에 표시되는 참조 언어(이미 main.ftl 파일이 있는 언어 중에서).
번역할 언어	해당 국가의 언어 중에서 선택하는 대상 언어.

필드	역할
글꼴	이 언어에 사용되는 글꼴. "찾아보기..." 버튼은 assets/fonts에 위치한 파일 선택기를 엽니다.
Google Fonts	브라우저에서 fonts.google.com 사이트를 엽니다.

중요한 제약: 글꼴은 assets/fonts 폴더에 설치되어 있어야 합니다. 다른 곳에 있는 글꼴은 거부되는데, 프로그램이 그 폴더를 기준으로 한 상대 경로로 글꼴을 로드하여 다음 시작 시 찾지 못하기 때문입니다. 따라서 글꼴이 없으면 Google Fonts에서 다운로드하여 선택하기 전에 그곳에 복사해야 합니다.

마찬가지로 ISO 639-3 코드가 있는 언어만 번역할 수 있습니다: 번역 폴더의 이름을 짓고 언어를 그 글꼴 및 참조 데이터에 연결하는 것이 바로 이 코드입니다. 그렇지 않으면 저장 버튼이 비활성화됩니다.

번역 표 — 세 개의 열:

열 1	열 2	열 3
키(편집 불가)	표시 언어의 레이블	번역 입력 필드

토큰 보호: 번역해서는 안 되는 요소($MODNAME$ 같은 매크로, { $num } 같은 변수)는 열 2에 그대로 나타나지 않으며 자동으로 보존됩니다.

- 텍스트의 시작이나 끝에 있는 토큰: 숨겨졌다가 저장 시 다시 삽입됩니다.
- 텍스트 중간에 있는 토큰: 유지하고 재배치할 수 있도록 간결한 표시 [[1]], [[2]]...로 바꿉니다.

애플리케이션 이름(app-title 키)은 의도적으로 제외되어 번역할 수 없습니다.

키보드 탐색 — 표는 마우스 없이 조작할 수 있습니다(부록 B 참조): Up/Down 방향키와 Page Up/Page Down은 행을 스크롤하고, Home/End는 첫 번째와 마지막으로 이동하며, Enter는 강조된 행의 입력 필드에 커서를 둡니다. 편집을 시작하면 Tab은 다음 필드로, Shift+Tab은 이전 필드로 이동하므로, 키보드를 떠나지 않고 번역을 연이어 입력할 수 있습니다. Page Up/Page Down으로 이동하는 행 수는 창 높이에 맞게 자동으로 조정됩니다.

저장 — "저장" 버튼은 파일 assets/locales/<language>/main.ftl을 기록하며, 그 앞에 Fluent 주석으로 작성된 메타데이터 블록이 붙습니다(따라서 파일은 완벽하게 유효한 상태로 유지됩니다):

```
# XIMOD Architect - translation metadata
# @language = fra
# @font = Noto_Sans/static/NotoSans-Regular.ttf
# @langname = Français
# @author = Patrice
```

헤더는 언어만 기술합니다: 같은 언어가 여러 국가에서 사용되기 때문에(독일어는 독일뿐 아니라 스위스에서도 쓰입니다) 국가 정보는 담지 않습니다. 국가는 선택한 플래그로 결정되며, XIMOD는 Countries.json과 CountryLanguages.json으로부터 국가 → 언어 매핑을 처리합니다.

엔도님 필드 채우기 — 국가(플래그)와 언어가 선택되면 XIMOD가 두 필드를 채웁니다:

- 국가 명칭: 선택한 국가에 대해 항상 Countries.json에서 확인됩니다. 따라서 플래그를 따릅니다(독일어는 독일의 경우 "Deutschland", 스위스의 경우 "Schweiz"를 보여 줍니다).
- 언어 명칭: 파일 헤더(# @langname)가 있을 때는 거기에서 가져옵니다. 없을 경우 — 헤더가 없는 오래된 파일이거나 아직 번역되지 않은 언어 — Languages.json에서 가져옵니다.

두 필드는 편집 가능한 상태로 유지됩니다: 사용자가 거기에서 수정할 수 있으며, 수정 사항은 저장 시 참조 파일에 전파됩니다.

저장은 참조 데이터를 갱신합니다: Languages.json의 언어 엔도님과 글꼴, 그리고 Countries.json의 국가 엔도님(국가 / 언어 쌍에 대해). 해당 국가에 대해 그 언어가 아직 거기에 없었다면 항목이 생성됩니다.

헤더가 더 이상 국가를 담지 않으므로, XIMOD는 main.ftl과 함께 탭으로 구분된 작은 파일 countryEndonyms.tsv를 기록합니다. 이 파일은 해당 언어에 대한 국가 엔도님을 "국가 코드, 언어 코드, 엔도님" 형태로 국가당 한 줄씩 기록합니다. 언어 폴더만 있을 때 국가 엔도님을 복구할 수 있게 하는 것이 바로 이 파일입니다.

번역 보내기 — "보내기..." 버튼은 번역 파일(main.ftl, 그 언어 폴더 안에)과, 있을 경우 함께 제공되는 countryEndonyms.tsv 파일을 담은 ZIP 아카이브를 만듭니다. XIMOD는 아카이브 폴더를 연 다음 미리 채워진 이메일을 준비합니다. 언어별 정보(언어, 글꼴, 언어 엔도님, 제작자)는 main.ftl의 헤더에 담겨 전달되며, 국가 엔도님은 countryEndonyms.tsv에 담겨 전달됩니다. Languages.json이나 Countries.json은 첨부되지 않습니다 — 실제로 사용된 코드만 전송되며, 이는 ISO 639-3 코드의 이용 약관을 준수합니다. 사용자가 직접 파일을 첨부하여 보냅니다: 명시적인 동작 없이는 어떤 데이터도 컴퓨터를 떠나지 않습니다.

> 참고 — 번역이 완료되었을 때만 패키지를 보내십시오: 번역되지 않은 채로 남은 키는 빈 값으로 저장되며, 이후 인터페이스에서 텍스트 없이 표시됩니다.

4.10 XML 편집기

프로젝트 → XML 편집기 → info.xml 또는 ModuleConfig.xml을 통해 접근할 수 있는 이 도구는 고급 사용자를 위한 것으로, 생성된 XML을 보고 직접 수정할 수 있게 합니다.

두 번째 화면을 포함하여 자유롭게 이동하고 크기를 조정할 수 있는 독립 창으로 열립니다.

두 가지 모드

- *읽기 전용(기본값): XML이 구문 강조와 내어쓰기 들여쓰기로 표시됩니다 — 너무 긴 줄은 줄바꿈되어 줄 시작 아래에 정렬되며, Notepad++ 방식과 같습니다. 메인 창은 계속 사용할 수 있습니다.*
- *편집("편집" 버튼): 텍스트가 편집 가능해지고 메인 창의 탭이 잠겨, 하나의 소스가 권위를 갖게 됩니다.*

구문 강조: 태그, 속성 이름, 따옴표로 묶인 값, 주석, 선언, 텍스트가 밝은 테마나 어두운 테마에 적합한 색으로 구분됩니다.

줄 번호 매기기: 여백 영역에 번호가 표시됩니다. 줄바꿈의 연속 줄은 비어 있는 상태로 유지됩니다.

실시간 검증: 입력하는 동안 XML이 지속적으로 확인됩니다. 녹색 표시기는 올바른 형식의 문서임을 확인해 주며, 오류가 있을 경우 빨간 메시지가 줄과 열을 명시하고 잘못된 토큰에 밑줄이 그어집니다.

변경 적용: "적용" 버튼은 텍스트를 다시 파싱하고 프로젝트를 갱신합니다. 오류가 있을 경우 변경이 거부되고 텍스트는 편집 가능한 상태로 유지됩니다. "취소" 버튼은 프로젝트에서 텍스트를 다시 생성합니다.

4.11 글꼴 관리

XIMOD Architect는 나열된 언어의 모든 문자 체계를 포괄하는 Noto 글꼴 세트를 assets/fonts에 담아 제공합니다.

이 글꼴들은 동적으로 로드됩니다: 언제든 실제로 필요한 것, 즉 인터페이스 언어의 글꼴과 선택한 국가의 언어 글꼴만 설치됩니다. 대부분의 국가에서는 단일 글꼴로 부하가 낮게 유지됩니다.

이 메커니즘은 비라틴 문자(Thai, Burmese, Amharic, Hmong 등)의 올바른 표시를 보장하며, 그렇지 않으면 빈 사각형으로 나타날 것입니다. 이는 인터페이스뿐 아니라 "속성" 창과 번역 편집기에도 적용됩니다: 거기에 표시되는 모든 언어의 글꼴이 로드되어 어떤 이름이나 레이블도 사각형으로 나타나지 않습니다.

XIMOD Architect는 비라틴 문자를 포함하여 전체 문자열 세트를 각각 포괄하는 32개의 인터페이스 번역을 담아 제공합니다: Japanese(jpn), Simplified Chinese(zho), Korean(kor), Russian(rus). English(eng)와 French(fra)는 수작업으로 유지되는 참조 번역이며, 나머지는 원어민이 번역 편집기를 통해 검토할 수 있는 기여로부터 나온 것입니다.

각 언어에 연결된 글꼴은 Languages.json에 표시되며 번역 편집기에서 변경할 수 있습니다.

4.12 두 개의 FOMOD 병합

파일 → FOMOD 병합… 명령을 사용하면 두 설치 프로그램의 설치 내용을 합쳐 결합할 수 있습니다. 현재 열려 있는 프로젝트가 수신자 역할을 하므로, 미리 생성하거나 열어 두어야 하며 그렇지 않으면 명령이 비활성화된 상태로 유지됩니다.

클릭하면 파일 선택기가 열립니다: 통합할 FOMOD(제공자)의 ModuleConfig.xml 파일을 선택하십시오. 그 단계, 필수 파일, 조건부 설치가 수신자 프로젝트의 것들 끝에 추가됩니다. 수신자 프로젝트의 메타데이터 — 이름, 제작자, 버전, 카테고리, 헤더 이미지 — 는 보존됩니다: 설치 내용만 가져옵니다.

병합 후 프로젝트는 수정됨으로 표시됩니다: 결과를 FOMOD 파일에 기록하려면 저장하는 것(Ctrl+S)을 잊지 마십시오. 병합은 여러 FOMOD를 차례로 통합하기 위해 반복할 수 있습니다.

> 참고 — 제공자의 ModuleConfig.xml 파일만 읽히며, 그 자체의 메타데이터(info.xml)는 가져오지 않습니다. 병합 후에는 결합된 단계 전반에 걸쳐 우선순위와 조건이 일관성을 유지하는지 확인하십시오.

4.13 배포 아카이브 내보내기

파일 → 배포 아카이브 내보내기… 명령은 단일 작업으로 업로드할 준비가 된 아카이브(Nexus Mods 또는 다른 모드 관리자)를 생성합니다. XIMOD는 먼저 FOMOD(fomod/info.xml 및 fomod/ModuleConfig.xml)를 루트 디렉터리에 기록한 다음, 전체 폴더를 ZIP 형식으로 압축합니다. 루트 디렉터리가 정의되지 않은 한 명령은 비활성화됩니다.

선택기는 "<name>-<version>.zip" 형태의 기본 이름(금지된 문자를 제거함)을 제공하며, 저장하기 전에 변경할 수 있습니다. 그런 다음 상태 표시줄에 기록된 파일 수와 아카이브 경로가 표시됩니다.

아카이브 구성 — fomod/ 하위 폴더와 모드 파일이 모드 관리자가 기대하는 그대로 아카이브 루트에 놓입니다. 내부 경로는 슬래시 "/"를 사용합니다(크로스 플랫폼 호환성).

> 참고 — 불필요한 파일(.git, .DS_Store, Thumbs.db, desktop.ini 등)과 아카이브 자체는 자동으로 제외됩니다. 따라서 내보내기 전에 모든 모드 파일이 실제로 루트 디렉터리 아래에 있는지 확인하십시오.

4.14 "속성" 창

도구 → 속성…을 통해 열리는 이 독립 창은 — 두 번째 화면을 포함하여 이동하고 크기를 조정할 수 있으며 — 내장된 국가 / 언어 데이터베이스의 읽기 전용 탐색기를 제공합니다. 여기서는 데이터베이스를 수정할 수 없습니다. 이것은 조회 도구입니다.

국가 탭: 선택한 국가에 대해 창은 그 플래그, 프랑스어 및 영어 이름, 공용 언어(엔도님, 글꼴, ISO 639-3 / 639-1 코드와 함께), 그리고 거기서 사용되는 모든 언어의 목록을 보여 줍니다.

언어 탭: 선택한 언어에 대해 창은 그 ISO 코드, 글꼴을 보여 주고, 이 언어가 사용되는 국가를 역방향으로 조회합니다. "사용 지역" 목록은 각 국가를 설정에서 선택한 언어로 이름 짓습니다(플래그 → 국가 → 언어): 해당 언어가 그곳에서 공용어일 때는 Countries.json의 그 언어 엔도님(이탈리아어 사용자의 경우 "Confederazione Svizzera"), 그렇지 않으면 CountryNames.json의 번역된 이름(독일어 사용자의 경우 "Südafrika"), 그리고 최후의 수단으로 영어 이름.

키보드 탐색 — 국가 목록과 언어 목록 모두 키보드로 조작할 수 있습니다(부록 B 참조): Up/Down 방향키, Page Up/Page Down(그 단계는 창 높이에 맞게 조정됨), Home/End, 그리고 강조된 항목을 선택하는 Enter. Tab과 Shift+Tab은 플래그에 절대 놓이지 않고 목록 내에서 선택을 이동합니다. 각 "필터:" 필드에는 오른쪽에 지우기 아이콘(빨간 십자)이 있으며, 창은 Esc로 닫힙니다.

4.15 명령줄 모드 (CLI)

인수와 함께 실행하면 XIMOD Architect는 어떤 창도 열지 않습니다: 요청된 명령을 실행한 다음 종료 코드(성공 시 0, 실패나 검증 이상 시 0이 아닌 값)와 함께 종료합니다. 이 모드를 사용하면 스크립트나 지속적 통합 파이프라인에서 하나 이상의 FOMOD의 재빌드와 패키징을 자동화할 수 있습니다.

> 참고 — Windows에서는 실행 파일이 "windows" 애플리케이션으로 컴파일되므로, 메시지를 출력하기 위해 자신을 실행한 콘솔에 연결됩니다(AttachConsole을 통해). 출력을 보려면 명령 프롬프트(cmd 또는 PowerShell)에서 실행하십시오. CLI 명령을 더블 클릭하면 아무것도 표시되지 않습니다.

사용 가능한 명령은 다음과 같습니다:

명령	효과
ximod-architect validate <root>	디렉터리의 FOMOD를 검증합니다(프로젝트 확인 및 ModConfig 5.0 스키마 부합). 각 이상의 위치가 표시됩니다. 이상이 발견되면 0이 아닌 종료 코드.
ximod-architect build <root>	기존 프로젝트로부터 fomod/info.xml과 fomod/ModuleConfig.xml을 (다시) 기록합니다.
ximod-architect package <root> [-o FILE]	FOMOD를 기록한 다음 .zip 배포 아카이브를 빌드합니다. -o 옵션으로 출력 경로를 지정할 수 있으며, 그렇지 않으면 아카이브가 현재 디렉터리에 생성됩니다.
ximod-architect help	도움말(명령 목록)을 표시합니다.
ximod-architect version	버전 번호를 표시합니다.

여기서 <root>는 모드의 루트 디렉터리(fomod 하위 폴더를 포함하는 — 또는 포함하게 될 — 폴더)를 가리킵니다.

4.16 업데이트 확인

기본적으로 XIMOD Architect는 시작할 때 하루에 한 번 더 새로운 버전이 있는지 확인합니다. 있으면 메뉴 바 아래에 다운로드 링크(Nexus 및 GitHub)와 함께 눈에 띄지 않는 배너가 나타납니다. 도움말 → 업데이트 확인…에서 언제든지 확인할 수도 있습니다. 이것은 설정 창("시작 시 업데이트 확인")에서 끌 수 있습니다.

개인정보: 이 확인은 애플리케이션이 하는 유일한 네트워크 요청입니다. 어떤 개인 데이터도 보내지 않습니다 — 최신 버전 번호를 읽기 위해 공개 GitHub 릴리스 페이지로 단 하나의 보안(HTTPS) 요청만 보냅니다. XIMOD는 스스로 어떤 것도 다운로드하거나 설치하지 않습니다. 다운로드 링크를 따를지는 항상 사용자가 선택합니다.

5. FOMOD 태그 참조

이 섹션은 XIMOD Architect가 실제로 생성하는 태그를 설명합니다. 두 파일은 모드의 fomod 하위 폴더에 UTF-8(바이트 순서 표시 포함)로 기록됩니다: info.xml은 네 칸 들여쓰기로, ModuleConfig.xml은 탭으로 들여쓰기됩니다. 읽을 때는 바이트 순서 표시 덕분에 인코딩이 자동으로 검출되므로, 다른 도구로 생성된 UTF-8과 UTF-16(LE 또는 BE) 파일을 모두 열 수 있습니다.

5.1 info.xml 파일

<fomod> 루트 아래의 메타데이터입니다. 요소는 아래 표의 순서로 기록되며, "비어 있지 않으면"으로 표시된 것은 해당 필드가 비어 있을 때 생략됩니다.

태그	내용	기록
<Name>	모드 이름.	항상
<Author>	제작자.	비어 있지 않으면
<Version>	버전.	비어 있지 않으면
<Website>	모드 페이지의 URL.	비어 있지 않으면
<Description>	설명.	비어 있지 않으면
<Groups>	모드의 카테고리.	항상
<Game>	선택한 게임의 식별자. XIMOD 고유 확장(FOMOD 사양 외부)으로, 모드 관리자가 무시합니다.	비어 있지 않으면

완전한 예시:

```
<?xml version="1.0" encoding="utf-8"?>
<fomod>
    <Name>My Awesome Mod</Name>
    <Author>Patrice</Author>
    <Version>1.0.0</Version>
    <Website>https://www.nexusmods.com/skyrimspecialedition/mods/12345</Website>
    <Description>A sample mod for the manual.</Description>
    <Groups>Player homes</Groups>
    <Game>skyrimSpecialEdition</Game>
</fomod>
```

5.2 ModuleConfig.xml 파일

ModConfig5.0.xsd 스키마를 참조하는 <config> 루트 아래의 설치 로직입니다. 블록은 다음 순서로 나타납니다: 모듈 이름, 헤더 이미지, 필수 파일, 설치 단계, 조건부 설치.

루트 및 헤더

태그 / 속성	역할
<config>	루트. xmlns:xsi 및 xsi:noNamespaceSchemaLocation 속성(ModConfig5.0.xsd 스키마)을 담고 있습니다.
<moduleName>	마법사 상단에 표시되는 모듈 이름.
<moduleImage path="…"/>	헤더 이미지(빈 태그, 이미지가 정의된 경우에만 기록됨).

파일 및 폴더

태그 / 속성	역할

태그 / 속성	역할
`<file source="…" destination="…" priority="…"/>`	파일 복사. source = 아카이브 내 경로(루트 기준); destination = Data 아래의 대상; priority = 설치 순서.
`<folder source="…" destination="…" priority="…"/>`	전체 폴더 복사(동일한 속성).
`<requiredInstallFiles>`	체계적으로 설치되는 파일의 컨테이너.
`<files>`	옵션이나 조건부 패턴에 연결된 파일의 컨테이너.

단계 및 그룹

태그 / 속성	역할
`<installSteps order="Explicit">`	단계의 컨테이너(정의된 순서가 보존됨).
`<installStep name="…">`	마법사의 한 페이지.
`<visible>`	단계의 표시 조건(<dependencies>를 포함함).
`<optionalFileGroups>`	단계의 옵션 그룹의 컨테이너.
`<group name="…" type="…">`	그룹; type = 선택 규칙(아래 참조).
`<plugins order="Explicit">`	그룹의 옵션의 컨테이너.

<group>의 type 속성 값:

type	제약
정확히 하나(필수) — SelectExactlyOne	정확히 하나의 옵션.
최대 하나 — SelectAtMostOne	최대 하나의 옵션.
개수 제한 없음 — SelectAny	임의의 개수.
모두(선택 불가) — SelectAll	모두, 강제됨.
최소 하나 — SelectAtLeastOne	최소 하나.

옵션 (플러그인)

태그 / 속성	역할
`<plugin name="…">`	사용자에게 제공되는 옵션.
`<description>`	옵션의 설명 텍스트(항상 기록됨).
`<image path="…"/>`	옵션을 설명하는 이미지(정의된 경우).
`<conditionFlags>`	옵션이 선택되면 설정되는 플래그의 컨테이너.
`<flag name="…">value</flag>`	플래그: 이름은 속성에, 값은 내용에.
`<typeDescriptor>`	옵션의 유형을 기술(단순 또는 조건부).
`<type name="…"/>`	단순 유형(종속성 패턴이 정의되지 않은 경우).
`<dependencyType>`	동적 유형(패턴이 존재하는 경우).
`<defaultType name="…"/>`	어떤 패턴도 적용되지 않을 때 적용되는 기본 유형.

<type> / <defaultType>의 값:

유형	마법사에서의 효과
선택 — Optional	선택 해제된 옵션, 자유롭게 활성화 가능.
필수 — Required	필수 옵션, 선택 해제 불가.
권장 — Recommended	기본적으로 선택된 옵션, 선택 해제 가능.
사용 불가 — NotUsable	비활성화된 옵션(선택 불가).
사용 가능할 수 있음 — CouldBeUsable	사용 가능하지만 잠재적으로 위험한 것으로 표시됨.

종속성 및 조건

태그 / 속성	역할
`<dependencies operator="…">`	조건 그룹; operator = And 또는 Or.
`<fileDependency file="…" state="…"/>`	파일에 대한 조건; state = Active, Inactive 또는 Missing.
`<flagDependency flag="…" value="…"/>`	플래그와 그 기대 값에 대한 조건.
`<conditionalFileInstalls>`	조건부 설치 섹션.
`<patterns> / <pattern>`	패턴 목록; 각 패턴은 <dependencies>를 <files>와(또는, typeDescriptor 내에서는 <type>과) 연결합니다.

6. ModuleConfig.xml 파일 상세 설명

5장에서는 태그를 참조 표로 제시했습니다. 이 장에서는 그것들을 하나씩 평이한 언어로 살펴보며, 각각이 무엇을 하는지, 어떤 옵션을 받아들이는지, 그리고 무엇보다 이 옵션들이 모드 사용자가 보게 될 설치 마법사에 어떤 구체적인 효과를 주는지 설명합니다. XML에 대한 사전 지식은 필요하지 않습니다: XIMOD Architect가 이 파일을 대신 작성해 줍니다. 그것을 이해하는 것은 더 신뢰할 수 있는 설치 프로그램을 설계하는 데 도움이 될 뿐입니다.

몇 가지 용어. XML 파일은 꺾쇠괄호로 둘러싸인 "태그"로 이루어집니다. 여는 태그 <step>은 닫는 태그 </step>와 짝을 이루며, 둘 사이에 쓰인 것은 그 태그에 속합니다. 태그는 "속성", 즉 이름="값" 형태로 작성된 설정을 가질 수 있습니다. 따라서 <group name="Resolution" type="SelectExactlyOne">는 두 개의 속성을 가진 "group" 태그입니다. 끝으로 내용이 없는 태그는 <file … />처럼 자체적으로 닫힙니다(끝의 /에 유의하십시오).

6.1 설치 프로그램이 파일을 읽는 방식

사용자가 당신의 모드를 설치하면, 그들의 관리자(Vortex, MO2, NMM)가 ModuleConfig.xml을 위에서 아래로 읽어 마법사를 구성합니다: 일련의 페이지(단계)로, 각각은 선택할 옵션 그룹을 제공합니다. 마지막에 관리자는 선택한 옵션에 해당하는 모든 파일을 모아 게임에 복사합니다. 유념해야 할 세 가지 개념:

- 순서가 중요하다: 단계는 파일에 나타나는 순서대로 표시됩니다;
- 선택한다고 바로 설치되는 것은 아니다: 옵션을 선택하면 선택지만 기록될 뿐이며, 실제 파일 복사는 맨 마지막에 일어납니다;
- 조건은 모든 것을 조절할 수 있다: 단계를 숨기거나, 옵션의 상태를 바꾸거나, 추가 파일의 설치를 유발하는 것 — 이것이 플래그와 종속성의 역할입니다(§ 6.6).

6.2 루트와 헤더

<config> — 루트. 전체 구성은 단일 <config>…</config> 태그 안에 들어갑니다. 이것은 검증 스키마(ModConfig 5.0)를 지정하는 두 개의 기술적 속성(xmlns:xsi 및 xsi:noNamespaceSchemaLocation)을 담고 있습니다. 당신이 이것들을 건드릴 필요는 전혀 없습니다: XIMOD는 항상 동일하게 기록합니다.

<moduleName> — 마법사 상단에 표시되는 모드 이름. 모드 정보 탭에서 입력한 것입니다.

<moduleImage path="…"/> — 마법사 상단에 표시되는 배너 이미지. path 속성은 모드 루트 기준 상대 위치(예: fomod\header.png)를 나타냅니다. 태그는 비어 있습니다(자체적으로 닫힙니다). 이미지가 없으면 그냥 생략됩니다.

6.3 파일 설치

두 개의 태그가 게임에 복사될 것을 기술합니다: <file>은 단일 파일을 복사하고, <folder>는 전체 내용이 담긴 폴더 하나를 복사합니다. 둘 다 동일한 세 가지 속성을 받습니다:

속성	역할	예시
source	루트 기준, 당신의 모드 아카이브 내 요소의 경로.	tex\2k
destination	게임의 Data 폴더 기준, 요소가 놓여야 하는 위치.	textures
priority	충돌 시 설치 순서(아래 참조).	0

priority 속성과 그 영향. 이것은 두 옵션이 같은 위치에 파일을 설치할 때 복사 순서를 정하는 숫자입니다. 규칙: 낮은 우선순위가 먼저 설치되고, 높은 것이 나중에 설치됩니다 — 그리고 마지막에 기록된 것이 이기므로, 더 높은 우선순위가 더 낮은 것을 "덮어씁니다". 예시: 기본 옵션이 texture를 priority="0"으로 설치하고 "고해상도" 옵션이 같은 texture를 priority="1"로 설치하면, 옵션의 표시 순서와 관계없이 고해상도 버전이 남습니다. 의도적으로 조정할 충돌이 없는 한 어디서나 0으로 두십시오.

> 참고 — destination은 기본적으로 비어 있으며, 이 경우 항목이 게임의 Data 폴더 루트에 설치됩니다. 항목이 다른 곳으로 가야 할 때마다 대상 셀을 클릭하여 명시적인 경로(Data 기준)를 입력하십시오. 소스 폴더가 이미 게임의 구조를 그대로 반영하고 있을 때는 빈 destination이 올바른 선택입니다.

이 태그들은 세 곳에 나타납니다: <requiredInstallFiles>(항상 설치되는 파일), 옵션의 <files>(옵션이 선택되면 설치됨), 그리고 조건부 패턴의 <files>(조건이 충족되면 설치됨).

6.4 단계, 그룹 및 옵션

이것은 마법사의 핵심입니다. 계층 구조는 간단합니다: 단계는 그룹을 포함하고, 그룹은 옵션을 포함합니다.

<installSteps order="Explicit">는 모든 단계를 모읍니다. order="Explicit"은 "내가 작성한 순서를 지키라"는 뜻입니다. 마법사의 각 페이지는 <installStep name="...">이며, 그 name 속성은 페이지 상단에 표시되는 제목입니다.

<visible> — 단계를 조건부로 표시. 특정 조건이 참일 때만 단계를 표시할 수 있습니다. 조건은 <visible> 태그에 넣습니다(§ 6.6 참조). <visible>이 없으면 단계는 항상 표시됩니다. 예시: 사용자가 앞서 "expert installation"을 선택한 경우에만 "advanced settings" 단계를 표시.

<optionalFileGroups>는 단계의 그룹을 담고 있습니다. 각 <group name="..." type="...">에는 이름(섹션 제목)과, 무엇보다 선택 규칙을 정하는 type — 즉 사용자가 무엇을 선택할 수 있는지 — 이 있습니다:

type	사용자가 보고 할 수 있는 것
정확히 하나(필수) — SelectExactlyOne	라디오 버튼: 하나의 옵션을, 오직 하나만 선택해야 합니다. 0개도 2개도 아닙니다. 배타적 변형(단일 texture 해상도)에 이상적입니다.
최대 하나 — SelectAtMostOne	위와 같지만, 아무것도 선택하지 않을 수도 있습니다: "옵션 하나 또는 없음".
개수 제한 없음 — SelectAny	독립적인 체크박스: 없음부터 전부까지 원하는 대로 선택합니다. 쌓을 수 있는 모듈에 가장 흔한 경우입니다.
최소 하나 — SelectAtLeastOne	체크박스이지만, 계속하려면 최소 하나를 선택해야 합니다. 빈손으로 넘어가는 것을 방지합니다.
모두(선택 불가) — SelectAll	모든 옵션이 자동으로 선택되고 잠깁니다: 사용자가 선택 해제할 수 없습니다. 필수 파일을 설명하면서 제시하는 데 사용됩니다.

그룹 안에서 <plugins order="Explicit">는 옵션을 담고 있으며, 각각은 <plugin name="...">입니다(여기서 "plugin"이라는 단어는 .esp 파일이 아니라 마법사의 옵션을 뜻합니다). 그 name은 사용자가 선택하는 레이블입니다. 옵션은 다음으로 구성됩니다:

- <description>: 옵션이 선택되었을 때 오른쪽에 표시되는 텍스트 — 거기에 그 효과를 명확하게 기술하십시오;
- <image path="..."/>: 설명과 함께 표시되는 삽화(선택 사항);
- <conditionFlags>: 옵션이 선택되면 설정되는 플래그(§ 6.6);

- <files>: 옵션이 선택되면 설치되는 파일;
- <typeDescriptor>: 옵션의 기본 상태(§ 6.5).

6.5 옵션의 유형 및 동작

각 옵션에는 마법사에서의 기본 상태를 결정하는 "유형"이 있습니다: 선택됨 또는 아님, 편집 가능 또는 아님. 이것은 <typeDescriptor>로 기술됩니다. 단순한 경우에는 단일 <type name="..."/>를 담고 있습니다. 다섯 가지 값과 그 효과:

type	마법사에서의 효과
선택 — Optional	기본적으로 선택 해제됨, 자유롭게 선택하고 해제할 수 있음. 일반적인 경우.
권장 — Recommended	기본적으로 선택됨, 하지만 사용자가 선택 해제할 수 있음. 권장하는 선택에 사용.
필수 — Required	선택되고 잠김: 선택 해제할 수 없음. 필수 구성 요소에 사용.
사용 불가 — NotUsable	선택 해제되고 잠김: 선택할 수 없음. 현재 맥락에서 사용할 수 없는 옵션임을 나타냄.
사용 가능할 수 있음 — CouldBeUsable	선택 가능하지만, 잠재적으로 문제가 될 수 있는 것으로 표시됨(흔히 경고와 함께).

동적 유형: <dependencyType>. 옵션의 유형은 조건에 따라 바뀔 수도 있습니다. 고정된 <type> 대신, <defaultType name="..."/>(어떤 조건도 충족되지 않을 때의 기본 유형)와 <pattern> 목록을 담은 <dependencyType>을 작성합니다: 각 패턴은 조건을, 그것이 참일 때 적용할 <type>과 연결합니다.

예시: "ENB Ultra preset" 옵션은 기본적으로 Optional이었다가, 사용자가 동적 촛불을 활성화하면 Recommended가 되고, 필수 파일이 디스크에 없으면 NotUsable이 될 수 있습니다. 설치 프로그램을 "똑똑하게" 만드는 것이 바로 이 메커니즘입니다. 완전한 예시는 부록 D에 나옵니다. 섹션 7.5에서는 이 메커니즘을 사용하여 "프리셋"을 사용자가 여전히 조정할 수 있는 기본값으로 바꿉니다.

6.6 플래그, 종속성 및 조건부 설치

이 세 가지 개념이 FOMOD 조건 시스템을 이룹니다 — 설치 프로그램이 사용자의 선택에 반응할 수 있게 하는 것입니다.

플래그 — <conditionFlags>와 <flag>. 플래그는 옵션이 선택될 때 설정하는 작은 변수입니다. 옵션의 <conditionFlags> 태그는 하나 이상의 <flag name="...">value</flag>를 담고 있습니다: name은 플래그 이름이고, 태그 사이의 텍스트는 그 값입니다. 따라서 <flag name="res">4K</flag>는 옵션이 선택되는 즉시 "res" 플래그를 "4K"로 설정합니다. 플래그 자체로는 눈에 보이는 효과가 없습니다: 나중에 다른 곳에서 조건으로 재사용되는 기억의 역할을 합니다.

종속성 — <dependencies operator="...">. 종속성은 확인할 조건입니다. 이들은 <dependencies>에 묶이며, 그 operator 속성이 결합 방식을 말합니다: And(모든 조건이 참이어야 함) 또는 Or(하나만으로 충분함). 두 종류의 조건이 있습니다:

조건	확인하는 것

조건	확인하는 것
`<flagDependency flag="…" value="…"/>`	명명된 플래그가 정확히 지정된 값을 가지면 참. 이것이 앞서 내린 선택에 반응하는 방법입니다(예: "res" 플래그가 "4K"와 같음).
`<fileDependency file="…" state="…"/>`	사용자의 게임에 이미 존재하거나 존재하지 않는 파일의 상태에 따라 참.

파일 종속성의 state 속성은 세 가지 값을 가집니다:

state	의미
Active	파일이 게임에 존재하고 활성화되어 있음.
Inactive	파일이 존재하지만 비활성화되어 있음.
Missing	파일이 없음.

이 종속성은 세 가지에 쓰입니다: 단계의 표시 여부(<visible> 내), 옵션의 동적 유형(<dependencyType>의 <pattern> 내), 그리고 아래의 조건부 설치.

조건부 설치 — <conditionalFileInstalls>. 단계 뒤에 놓이는 이 섹션은 사용자가 직접 무언가를 선택하지 않아도 조건이 충족될 때만 파일을 설치합니다. 이것은 <pattern> 요소를 담고 있으며, 각각은 <dependencies>를 <files>와 연결합니다. 마법사 끝에 패턴의 조건이 참이면 그 파일이 설치됩니다.

단계 표시 여부와의 차이: <visible>은 페이지를 표시할지 결정하는 반면, <conditionalFileInstalls>는 보이지 않는 곳에서 파일을 설치할지 결정합니다. 전형적인 경우: 사용자가 특정 두 옵션을 선택한 경우에만(그래서 그 플래그가 둘 다 설정된 경우에만) 호환성 패치를 자동으로 설치.

> 참고 — 플래그는 종속성이 그것을 확인하기 전에 먼저 옵션에 의해(<conditionFlags>를 통해) 설정되어야 합니다. 따라서 단계에서 플래그를 정의한 다음, 조건부 패턴에서 그것을 확인하십시오.
>
> 참고 — 설계 규칙: 옵션이 단지 하나의 결정 — 프리셋, 프로필, "모듈" 스위치 — 만을 나타낸다면, 그 옵션은 플래그만 설정하고 다른 일은 하지 않게 하며, 복사는 그 플래그를 조건으로 하는 조건부 설치에 맡기십시오. 그러면 기능당 하나의 플래그가 유일한 정보 출처가 됩니다. 선택이 어떻게 조합되든, 프리셋과 직접 고른 옵션이 같은 파일을 두 번 설치하는 일은 결코 없습니다. 섹션 7.5에서 이 규칙을 적용합니다.

7. ModuleConfig.xml 예제

아래 예제는 가장 흔한 경우를 보여 줍니다. 이들은 XIMOD Architect가 실제로 생성하는 XML에 해당합니다(가독성을 위해 들여쓰기를 단순화함).

7.1 최소 예제 — 필수 파일

선택지가 없는 모드: 모든 파일이 직접 설치됩니다.

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>My Simple Mod</moduleName>
    <requiredInstallFiles>
        <file source="MyMod.esp" destination="MyMod.esp" priority="0"/>
        <folder source="textures" destination="textures" priority="0"/>
    </requiredInstallFiles>
</config>
```

7.2 배타적 선택 — SelectExactlyOne

단일 texture 해상도의 선택을 제공하는 단계. 2K 옵션이 기본적으로 권장됩니다.

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>HD Textures</moduleName>
    <moduleImage path="fomod\header.png"/>
    <installSteps order="Explicit">
        <installStep name="Texture resolution">
            <optionalFileGroups>
                <group name="Resolution" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="2K">
                            <description>2K textures (balanced).</description>
                            <files>
                                <folder source="2k" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Recommended"/>
                            </typeDescriptor>
                        </plugin>
                        <plugin name="4K">
                            <description>4K textures (high quality).</description>
                            <files>
                                <folder source="4k" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Optional"/>
                            </typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
    </installSteps>
</config>
```

7.3 조건부 설치 — 플래그

옵션이 선택될 때 플래그를 설정하고, 그 플래그가 활성화된 경우에만 파일이 설치됩니다.

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Conditional patch</moduleName>
    <installSteps order="Explicit">
        <installStep name="Options">
            <optionalFileGroups>
                <group name="Modules" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="Module A">
                            <description>Enables module A.</description>
                            <conditionFlags>
                                <flag name="ModuleA">On</flag>
                            </conditionFlags>
                            <files>
                                <folder source="A" destination="meshes" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Optional"/>
                            </typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
    </installSteps>
    <conditionalFileInstalls>
        <patterns>
            <pattern>
                <dependencies operator="And">
                    <flagDependency flag="ModuleA" value="On"/>
                </dependencies>
                <files>
                    <file source="patchA.esp" destination="patchA.esp" priority="0"/>
                </files>
            </pattern>
        </patterns>
    </conditionalFileInstalls>
</config>
```

7.4 한 페이지, 여러 단계 — SelectAny + 플래그

단일 다중 선택 단계는 플레이어가 선택하는 각 옵션에 대해 서로 다른 단계를 열 수 있습니다: 단계는 옵션에 매여 있지 않습니다. 각 옵션은 조건 플래그를 설정하고, 각 대상 단계는 그 플래그에 대한 표시 조건을 가집니다. A와 B를 선택하면 단계 A와 B가 표시되고, A, B, C를 선택하면 세 개의 단계가 표시됩니다.

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Modular Add-ons</moduleName>
    <installSteps order="Explicit">
        <!-- Step 1: the choices. Each ticked option sets its own flag. -->
        <installStep name="Choose modules">
            <optionalFileGroups>
                <group name="Modules" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="Module A">
                            <description>Enable module A.</description>
                            <conditionFlags>
                                <flag name="want_A">on</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <plugin name="Module B">
                            <description>Enable module B.</description>
                            <conditionFlags>
                                <flag name="want_B">on</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <!-- Modules C and D follow the same pattern (want_C, want_D). -->
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Step A: shown only when want_A is on. -->
        <installStep name="Module A options">
            <visible>
                <flagDependency flag="want_A" value="on"/>
            </visible>
            <optionalFileGroups>
                <group name="Module A" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="A - extra textures">
                            <description>Optional high-res textures for A.</description>
                            <files>
                                <folder source="a_textures" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Step B: shown only when want_B is on. -->
        <installStep name="Module B options">
            <visible>
                <flagDependency flag="want_B" value="on"/>
            </visible>
            <optionalFileGroups>
                <group name="Module B" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="B - extra meshes">
                            <description>Optional meshes for B.</description>
                            <files>
                                <folder source="b_meshes" destination="meshes" priority="0"/>
                            </files>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Steps for C and D have exactly the same shape, each guarded by its
             own <visible><flagDependency flag="want_C | want_D" value="on"/>. -->
    </installSteps></config>
```

7.5 사용자 지정 가능한 기본값을 갖춘 프리셋

"프리셋"은 여러 선택을 한 번에 대신하는 옵션입니다. 사용자는 두 가지를 모두 원합니다. 프리셋을 고르고 끝내는 것, 또는 프리셋을 고른 뒤에도 기능 한두 개를 조정하는 것입니다. 핵심은 결정과 설치를 분리하는 것입니다. 첫 페이지가 결정합니다 — 플래그를 설정합니다. 필요할 때 표시되는 기능 페이지는 사용자가 직접 고를 수 있게 합니다 — 다른 플래그를 설정합니다. 조건부 설치는 그 플래그에 따라 복사를 수행합니다.

단계 1은 프리셋을 제공하고, 기능마다 두 옵션으로 이루어진 그룹을 둡니다. "프리셋의 선택 따르기" 또는 "사용자 지정"입니다. 두 번째 옵션은 해당 기능의 페이지를 열며, 이 페이지는 그 "사용자 지정" 플래그가 on일 때만 표시됩니다(§ 7.4). 그 페이지에서 옵션은 동적 유형(§ 6.5)을 사용합니다. 선택한 프리셋이 설치했을 변형이 Recommended — 미리 선택됨 — 가 되고, 그렇지 않으면 "없음"이 권장되므로, 사용자는 프리셋의 변형을 유지하거나, 다른 변형으로 바꾸거나, 기능을 제거할 수 있습니다.

프리셋도 페이지 옵션도 <files>를 전혀 갖지 않습니다. 모든 복사는 <conditionalFileInstalls>에서 옵션마다 하나의 패턴으로 이루어집니다. 그 파일은 기능이 사용자 지정되었고 그 페이지에서 옵션이 선택된 경우, 또는 기능이 프리셋에 맡겨졌고 그 프리셋이 해당 옵션을 고른 경우에 설치됩니다. 바로 이 점이 이중 설치를 불가능하게 만듭니다. 각 기능에 대해 두 분기 중 정확히 하나만 적용되며, 따라서 파일 세트도 하나뿐입니다.

> 참고 — 플래그 하나에 기록자 하나. 모드 관리자는 플래그를 클릭할 때마다 갱신되는 표에 보관합니다. 옵션을 선택하면 그 플래그가 기록되고, 선택을 해제하면 제거되며, 마지막에 기록한 쪽이 이깁니다. 함께 선택할 수 있는 두 옵션이 같은 플래그를 기록하면 — 페이지의 플래그까지 설정하는 프리셋, 그 플래그를 초기화하는 "사용자 지정" 확인란 — 뒤로 돌아가 둘 중 하나를 다시 클릭할 때 다른 쪽이 여전히 선택된 것으로 표시하는 내용을 조용히 덮어씁니다. 따라서 각 플래그는 서로 배타적인 옵션들로 이루어진 하나의 그룹만 기록하게 하십시오. preset은 "프리셋" 그룹이, cust_A는 기능 A의 모드 그룹이, feat_A는 페이지 A의 옵션이 기록합니다. 그러면 클릭 순서가 어떻든 플래그는 항상 사용자가 보는 것과 일치합니다.

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>사용자 지정 가능한 기본값을 갖춘 프리셋</moduleName>
    <installSteps order="Explicit">
        <!-- 단계 1: 프리셋은 preset 플래그를 기록할 뿐, 다른 일은 하지 않습니다. -->
        <installStep name="설치 모드">
            <optionalFileGroups>
                <group name="프리셋" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="프리셋 A(권장 구성)">
                            <description>기능 A를 버전 2로, 기능 B를 버전 1로 설치합니다.</description>
                            <conditionFlags><flag name="preset">A</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <plugin name="프리셋 없음(직접 구성)">
                            <description>모든 기능을 직접 고릅니다.</description>
                            <conditionFlags><flag name="preset">none</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
                <!-- 기능마다 그룹 하나: 이 그룹만이 해당 기능의 "사용자 지정" 플래그를 기록합니다. -->
                <group name="기능 A" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="프리셋의 선택 따르기">
                            <description>프리셋이 설치하는 대로의 기능 A.</description>
                            <conditionFlags><flag name="cust_A">off</flag></conditionFlags>
                            <typeDescriptor><type name="Recommended"/></typeDescriptor>
```

```
                    </plugin>
                    <plugin name="사용자 지정">
                        <description>기능 A의 페이지를 엽니다.</description>
                        <conditionFlags><flag name="cust_A">on</flag></conditionFlags>
                        <typeDescriptor><type name="Optional"/></typeDescriptor>
                    </plugin>
                </plugins>
            </group>
            <!-- “기능 B”, “기능 C”...도 같은 두 옵션을 가지며 cust_B, cust_C를 기록합니다. -->
        </optionalFileGroups>
    </installStep>
    <!-- 단계 2: 기능 A. 사용자가 사용자 지정을 요청했을 때만 표시됩니다. -->
    <installStep name="기능 A">
        <visible><flagDependency flag="cust_A" value="on"/></visible>
        <optionalFileGroups>
            <group name="기능 A" type="SelectExactlyOne">
                <plugins order="Explicit">
                    <plugin name="없음">
                        <description>기능 A를 설치하지 않습니다.</description>
                        <conditionFlags><flag name="feat_A">none</flag></conditionFlags>
                        <typeDescriptor>
                            <dependencyType>
                                <defaultType name="Recommended"/>
                                <patterns>
                                    <pattern>
                                        <dependencies operator="And">
                                            <flagDependency flag="preset" value="A"/>
                                        </dependencies>
                                        <type name="Optional"/>
                                    </pattern>
                                </patterns>
                            </dependencyType>
                        </typeDescriptor>
                    </plugin>
                    <plugin name="버전 1">
                        <description>기능 A, 버전 1.</description>
                        <conditionFlags><flag name="feat_A">v1</flag></conditionFlags>
                        <typeDescriptor><type name="Optional"/></typeDescriptor>
                    </plugin>
                    <plugin name="버전 2">
                        <description>기능 A, 버전 2(프리셋의 선택).</description>
                        <conditionFlags><flag name="feat_A">v2</flag></conditionFlags>
                        <typeDescriptor>
                            <dependencyType>
                                <defaultType name="Optional"/>
                                <patterns>
                                    <pattern>
                                        <dependencies operator="And">
                                            <flagDependency flag="preset" value="A"/>
                                        </dependencies>
                                        <type name="Recommended"/>
                                    </pattern>
                                </patterns>
                            </dependencyType>
                        </typeDescriptor>
                    </plugin>
                </plugins>
            </group>
        </optionalFileGroups>
    </installStep>
    <!-- 기능 B와 C의 페이지도 정확히 같은 형태입니다. -->
</installSteps>
<!-- 파일은 이곳에서만 복사됩니다. 기능이 사용자 지정되었으면 페이지가, 그렇지 않으면 프리셋이 복사합니다. --
>
<conditionalFileInstalls>
    <patterns>
        <pattern>
            <dependencies operator="And">
                <flagDependency flag="cust_A" value="on"/>
                <flagDependency flag="feat_A" value="v1"/>
            </dependencies>
            <files><folder source="FeatureA\v1" destination="" priority="0"/></files>
        </pattern>
        <pattern>
            <dependencies operator="Or">
```

```
                    <dependencies operator="And">
                        <flagDependency flag="cust_A" value="on"/>
                        <flagDependency flag="feat_A" value="v2"/>
                    </dependencies>
                    <dependencies operator="And">
                        <flagDependency flag="cust_A" value="off"/>
                        <flagDependency flag="preset" value="A"/>
                    </dependencies>
                </dependencies>
                <files><folder source="FeatureA\v2" destination="" priority="0"/></files>
            </pattern>
            <!-- 기능 B도 마찬가지입니다. v1은 페이지 또는 프리셋 A가 설치합니다. -->
        </patterns>
    </conditionalFileInstalls>
</config>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>사용자 지정 가능한 기본값을 갖춘 프리셋</moduleName>
    <installSteps order="Explicit">
        <!-- 단계 1: 프리셋은 포함된 모든 기능의 플래그를 설정할 뿐, 다른 일은 하지 않습니다. -->
        <installStep name="설치 모드">
            <optionalFileGroups>
                <group name="프리셋" type="SelectAtMostOne">
                    <plugins order="Explicit">
                        <plugin name="프리셋 A(권장 구성)">
                            <description>기능 A를 버전 2로, 기능 B를 버전 1로 설치합니다.</description>
                            <conditionFlags>
                                <flag name="preset">on</flag>
                                <flag name="feat_A">v2</flag>
                                <flag name="feat_B">v1</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
                <group name="사용자 지정" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="기능 A 사용자 지정">
                            <description>기능 A의 페이지를 엽니다.</description>
                            <conditionFlags><flag name="cust_A">on</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <!-- “기능 B 사용자 지정”, “기능 C 사용자 지정”...은 cust_B, cust_C를 설정합니다. -->
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- 단계 2: 기능 A. 사용자가 사용자 지정을 요청했을 때만 표시됩니다. -->
        <installStep name="기능 A">
            <visible><flagDependency flag="cust_A" value="on"/></visible>
            <optionalFileGroups>
                <group name="기능 A" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="없음">
                            <description>기능 A를 설치하지 않습니다.</description>
                            <conditionFlags><flag name="feat_A">none</flag></conditionFlags>
                            <typeDescriptor>
                                <dependencyType>
                                    <defaultType name="Recommended"/>
                                    <patterns>
                                        <pattern>
                                            <dependencies operator="And">
                                                <flagDependency flag="preset" value="on"/>
                                            </dependencies>
                                            <type name="Optional"/>
                                        </pattern>
                                    </patterns>
                                </dependencyType>
                            </typeDescriptor>
                        </plugin>
                        <plugin name="버전 1">
                            <description>기능 A, 버전 1.</description>
                            <conditionFlags><flag name="feat_A">v1</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
```

```
                    <plugin name="버전 2">
                        <description>기능 A, 버전 2(프리셋의 선택).</description>
                        <conditionFlags><flag name="feat_A">v2</flag></conditionFlags>
                        <typeDescriptor>
                            <dependencyType>
                                <defaultType name="Optional"/>
                                <patterns>
                                    <pattern>
                                        <dependencies operator="And">
                                            <flagDependency flag="preset" value="on"/>
                                        </dependencies>
                                        <type name="Recommended"/>
                                    </pattern>
                                </patterns>
                            </dependencyType>
                        </typeDescriptor>
                    </plugin>
                </plugins>
            </group>
        </optionalFileGroups>
    </installStep>
    <!-- 기능 B와 C의 페이지도 정확히 같은 형태입니다. -->
</installSteps>
<!-- 파일은 각 기능 플래그의 최종 값에 따라 이곳에서만 복사됩니다. -->
<conditionalFileInstalls>
    <patterns>
        <pattern>
            <dependencies operator="And"><flagDependency flag="feat_A" value="v1"/></dependencies>
            <files><folder source="FeatureA\v1" destination="" priority="0"/></files>
        </pattern>
        <pattern>
            <dependencies operator="And"><flagDependency flag="feat_A" value="v2"/></dependencies>
            <files><folder source="FeatureA\v2" destination="" priority="0"/></files>
        </pattern>
        <pattern>
            <dependencies operator="And"><flagDependency flag="feat_B" value="v1"/></dependencies>
            <files><folder source="FeatureB\v1" destination="" priority="0"/></files>
        </pattern>
    </patterns>
</conditionalFileInstalls>
</config>
```

프로젝트 → 설치 프로그램 미리 보기…를 사용하면 각 조합 — 프리셋만, 프리셋과 "기능 A 사용자 지정", "기능 A 사용자 지정"만 — 을 차례로 살펴보고 게시하기 전에 최종 파일 목록을 확인할 수 있습니다(§ 4.11, § 8.19). 프로젝트 → FOMOD 유효성 검사는 충돌하는 대상을 보고하고 이 프리셋 / 사용자 지정 패턴을 인식합니다(§ 8.1).

설치 프로그램에서 뒤로 돌아가기. 모드 관리자는 사용자가 이미 본 페이지의 선택을 유지합니다. 사용자가 첫 페이지로 돌아가 다른 프리셋을 고르면, 이미 방문한 기능 페이지는 새 프리셋의 기본값 대신 이전의 선택을 유지합니다. 플래그마다 기록자가 하나이면 이는 무해합니다. 플래그는 항상 화면과 일치하므로, 사용자가 최종적으로 보는 것 — 프리셋에 맡긴 기능에 대해서는 프리셋, 사용자 지정한 페이지에서는 선택 표시 — 이 바로 설치되는 것입니다. 프리셋 마법사(§ 8.29)는 이 구조를 구축합니다. 프리셋이 페이지의 플래그까지 기록하던 이전 버전으로 만든 구조는 마법사를 다시 실행하면 다시 만들어집니다.

7.6 소개 페이지와 마무리 페이지

마법사를 그리는 것은 설치 프로그램이 아니라 모드 관리자입니다. 제작자는 단계, 그룹, 옵션의 이름, 설명, 이미지를 제어할 수 있을 뿐 그 밖의 것은 제어할 수 없습니다. 글꼴도, 색상도, 레이아웃도, 자유 텍스트 블록도 없습니다. 이 한도 안에서 환영 페이지와 마무리 페이지는 쉽게 만들 수 있습니다.

환영 페이지 — 첫 번째 단계를 Required 옵션 하나를 담은 SelectAll 유형의 단일 그룹으로 만드십시오. 그 설명에는 환영 문구를 담으며, 관리자는 이를 설명 영역에 표시합니다. 그

이미지에는 시각 요소 — 배너, 로고, 또는 원하는 서체로 그림처럼 렌더링한 텍스트 — 를 담습니다. 파일은 지정하지 않거나, 모드의 핵심 파일이 그곳에 나열되기를 원한다면 그 파일을 지정하십시오.

마무리 페이지 — 마지막 단계로 같은 구성을 사용합니다. 작별 메시지와, 필요하다면 다음에 할 일(로드 순서, 설치할 패치)을 담은 Required 옵션입니다. 설치 프로그램은 사용자의 선택을 요약해 표시할 수 없습니다. 이 형식에서는 한 페이지가 다른 페이지에서 내린 선택에 접근할 수 없기 때문입니다. 관리자는 곧 설치될 파일 목록을 자체적으로 보여 주며, 설치 프로그램 미리 보기(§ 4.11)는 제작자인 여러분에게 모든 시나리오에 대한 그 요약을 제공합니다.

```
<installStep name="환영합니다">
    <optionalFileGroups>
        <group name="이 설치 프로그램에 대하여" type="SelectAll">
            <plugins order="Explicit">
                <plugin name="Dawnlight에 오신 것을 환영합니다">
                    <description>Dawnlight를 다운로드해 주셔서 감사합니다. 다음 페이지에서 조명 프리셋, 선택
모듈, 호환성 패치를 고를 수 있습니다. 마지막 페이지 전까지는 아무것도 설치되지 않습니다.</description>
                    <image path="fomod\welcome.png"/>
                    <typeDescriptor><type name="Required"/></typeDescriptor>
                </plugin>
            </plugins>
        </group>
    </optionalFileGroups>
</installStep>
<!-- ... 실제 단계 ... -->
<installStep name="모두 완료">
    <optionalFileGroups>
        <group name="플레이하기 전에" type="SelectAll">
            <plugins order="Explicit">
                <plugin name="감사합니다!">
                    <description>Dawnlight.esp를 날씨 모드 뒤에 배치하십시오. 문제가 있으면 Nexus 페이지에
알려 주십시오. 즐겁게 플레이하십시오!</description>
                    <image path="fomod\thanks.png"/>
                    <typeDescriptor><type name="Required"/></typeDescriptor>
                </plugin>
            </plugins>
        </group>
    </optionalFileGroups>
</installStep>
```

환영 단계를 템플릿으로 저장하면(프로젝트 → 템플릿..., § 8.3) 모든 모드에서 재사용할 수 있습니다.

8. 버전 2 — 새 기능

아래 기능은 버전 2.0.0에서 도입되었습니다. 추가된 순서대로 제시되며, 2장부터 7장까지의 번호가 매겨진 섹션은 이미 이를 반영하고 있습니다. 섹션 8.26부터 8.29까지는 버전 2.1.0에서 추가되었습니다. 섹션 8.30부터 8.32까지는 버전 2.2.0에서 추가되었습니다.

8.1 대상 충돌 검출

프로젝트를 검증할 때(프로젝트 → FOMOD 유효성 검사), XIMOD는 이제 서로를 덮어쓰게 되는, 서로 다른 파일을 같은 대상 경로에 설치하는 옵션도 보고합니다. 함께 발생할 수 있는 충돌은 확실한 것으로 표시되고, 상호 배타적인 옵션 사이에서만 발생하는 충돌은 가능성 있는 것으로 표시됩니다.
충돌은 두 옵션이 실제로 함께 선택될 수 있을 때만 확실한 것으로 보고됩니다. 배타적 그룹(SelectExactlyOne / SelectAtMostOne) 외에도, XIMOD는 이제 플래그를 따라갑니다: 한 옵션이 플래그를 어떤 값으로 설정하고 다른 옵션이 그 플래그의 다른 값에 대해서만 표시되는 페이지에 있으며 — 첫 번째 옵션과 함께 선택 가능한 어떤 것도 그 다른 값을 설정할 수 없다면 — 이 쌍은 공존할 수 없으므로 충돌은 잠재적인 것으로만 나열됩니다. 전형적인 경우: "preset" 옵션과 "choose your own" 페이지의 대응 옵션.

8.2 7z 내보내기

배포 아카이브 내보내기… 명령은 이제 ZIP에 더해 7-Zip(.7z)을 제공합니다. 저장 대화 상자에서 형식을 선택하십시오. 7z는 보통 더 작은 다운로드를 생성합니다.

8.3 재사용 가능한 템플릿

프로젝트 → 템플릿…을 사용하면 선택한 단계를 재사용 가능한 템플릿으로 저장하고, 저장된 템플릿(단계, 그룹 또는 옵션)을 어떤 프로젝트에든 삽입할 수 있습니다. 템플릿은 애플리케이션의 구성 폴더 아래에 저장됩니다.

8.4 폴더에서 새로 만들기

파일 → 폴더에서 새로 만들기…는 폴더를 스캔하여 각 바로 아래 하위 폴더가 하나의 옵션이 되는 뼈대를 제안합니다. 루트에 있는 명백한 헤더 이미지는 자동으로 검출됩니다.

8.5 끌어다 놓기로 파일 지정

트리에서 옵션을 선택한 상태로 파일 탐색기(Windows 탐색기, Finder, Linux 파일 관리자)의 파일이나 폴더를 XIMOD 창으로 끌어다 놓으십시오. "파일 추가" / "폴더 추가"를 사용할 때와 똑같이 옵션의 소스 표에 추가되며, 여러 항목을 한 번에 놓을 수도 있습니다. 프로젝트 루트 폴더 아래의 경로만 허용되며, 나머지는 무시됩니다. 대규모 설치 프로그램에서는 이것이 소스를 지정하는 가장 빠른 방법입니다. 탐색기에서 기능 폴더를 한 번 연 다음, 각 하위 폴더를 해당 옵션에 끌어다 놓으십시오.

8.6 FOMOD 비교

프로젝트 → 비교…는 현재 프로젝트를 다른 FOMOD와 비교하여 추가, 제거 또는 변경된 사항(단계, 옵션, 파일, 메타데이터)을 나열합니다 — 변경 로그를 작성하는 데 편리합니다.

8.7 시각적 조건 편집기

프로젝트 → 조건 편집기…는 모든 조건 플래그에 대해 어떤 옵션이 그것을 설정하고 어떤 단계 표시 규칙, 조건부 설치, 종속성 패턴이 그것을 사용하는지 보여 줍니다. 설정되었지만 사용되지 않는(또는 사용되지만 설정되지 않는) 플래그는 강조 표시됩니다.

8.8 이미지 확인 및 최적화

검증은 이제 너무 크거나, 지원되지 않는 형식이거나, 읽을 수 없는 헤더/옵션 이미지에 대해 경고합니다. 모드 정보 탭에서 이미지 최적화는 헤더 이미지를 적절한 범위로 축소합니다.

8.9 명령줄 추가 사항

명령줄에 다음이 추가됩니다: inspect <plugin.esp>(플러그인의 마스터, 제작자, ESL 플래그를 출력); batch <manifest.json>(여러 FOMOD를 검증, 빌드, 패키징, 지속적 통합용); 그리고 폴더를 일반 BA2 아카이브로 묶는 ba2 <dir> <out.ba2> [game]. BA2 패키징은 실험적입니다 — 배포하기 전에 게임 내에서 아카이브를 확인하십시오.

8.10 백그라운드 내보내기 및 검증

배포 아카이브 내보내기는 더 이상 창을 멈추게 하지 않습니다: FOMOD가 기록되면 압축이 백그라운드에서 실행되고, 진행 창에 처리된 파일 수와 추가 중인 파일이 표시됩니다. 취소 버튼은 내보내기를 중단하고 부분 아카이브를 제거합니다. 마찬가지로, 검증의 디스크 확인(누락된 파일, 대상 충돌, 이미지)이 백그라운드에서 실행됩니다. 보고서 창은 완료될 때까지 "디스크의 파일을 확인하는 중…"을 표시하며, 그 사이에 프로젝트가 변경되었으면 경고합니다.

8.11 충돌 복구

XIMOD Architect가 예기치 않게 닫히면, 저장되지 않은 변경 사항이 있는 모든 프로젝트가 애플리케이션 구성 디렉터리 아래의 복구 폴더(crash_recovery/<date>/<mod name>/fomod/)에 기록됩니다. 다음 시작 시 상태 표시줄에 백업이 저장된 위치가 표시됩니다. 파일 → 폴더 열기…로 그 폴더를 열어 작업을 되찾으십시오.

8.12 더 명확해진 인터페이스

일상적인 성가신 점 여럿이 개선되었습니다. 아직 폴더가 없는 프로젝트를 저장하면 이제 아무것도 하지 않는 대신 저장할 위치를 묻고, 파일 → 다른 이름으로 저장… (Ctrl+Shift+S)은 프로젝트를 다른 폴더로 옮깁니다. 창 제목은 모드 이름과, 저장되지 않은 변경 사항이 있는 동안 ● 점을 표시합니다. 성공, 경고, 오류는 오른쪽 상단 모서리에 알림으로 나타나며(오류는 닫을 때까지 유지됨), 상태 표시줄은 마지막 메시지와 단계 및 옵션의 개수를 유지합니다(루트 폴더는 인스펙터의 작업 공간 섹션에, 모드 이름은 창 제목과 문서 탭에 표시됩니다). 루트 폴더 밖의 파일을 선택하면 이제 조용히 무시되는 대신 보고됩니다.
용어 — 인터페이스는 이제 FOMOD 형식이 plugin이라고 부르는 것(그룹에서 제공되는 선택지)에 대해 모든 언어에서 일관되게 "option"이라고 말합니다. "plugin"은 게임 플러그인 파일(.esp, .esm, .esl)에 한정하여 사용합니다. 테마 설정 "시스템 설정 따름"은 운영 체제를 실시간으로 따릅니다: OS를 밝은 모드와 어두운 모드 사이에서 전환하면 XIMOD도 다시 시작하지 않고 즉시 전환되며, 선택한 글꼴 크기는 둘 다에서 유지됩니다. 끝으로, 데이터셋(게임, 국가, 언어)이 스플래시 화면이 표시된 후가 아니라 표시되는 동안 읽히므로, 메인 창이 더 빨리 나타납니다.

8.13 기존 FOMOD 번역하기

프로젝트 → FOMOD 번역…은 설치 프로그램의 사용자 대면 문자열을 다른 언어로 번역하는 전용 창을 엽니다: 모드 이름과 설명, 단계, 그룹, 옵션의 이름, 그리고 옵션 설명. 플래그, 파일 경로, 유형은 절대 건드리지 않는데, 모드 관리자가 그것들을 문자 그대로 비교하기 때문입니다. 활성 프로젝트(저장되어 있어야 함)를 로드하거나 모드 폴더 열기…로 어떤 모드 폴더든 로드하십시오. 소스 언어와 대상 언어를 선택한 다음, 이름은 표에서 직접, 설명은 하단 패널에서 번역 열을 채우십시오(소스가 나란히 표시되며, 공백과 줄바꿈을 선택적으로 보이게 할 수 있습니다). 필터(미번역, 검토 필요, 문제 있음, 잠김), 검색 상자, 진행 막대, Ctrl+Enter(다음 미번역)가 진행 상황을 추적하는 데 도움이 됩니다. 제작자와 웹사이트는 기본적으로 잠겨 있습니다: 잠긴 문자열은 변경 없이 기록됩니다.

문제는 각 행에 표시됩니다: 번역에서 누락된 보호 토큰(파일 이름, URL, 문자 그대로의 \n 시퀀스), 이름의 줄바꿈, XML이 저장할 수 없는 문자, 소스와 동일한 번역, 또는 같은 소스 텍스트가 다른 곳에서 다르게 번역된 경우. 차단성 문제는 내보내기를 막습니다. 작업은 사이드카 파일 fomod/translations/<mod>.<language>.ximod-translation으로 저장(Ctrl+S)되며, 다음에 폴더를 열 때 창이 자동으로 다시 로드합니다. 폴더에서 업데이트는 FOMOD를 다시 읽어 새로운, 변경된, 이동된, 제거된 문자열을 보고하면서 기존 번역을 유지합니다(변경된 소스는 "검토 필요"로 표시됨).
번역된 파일 내보내기는 info.xml과 ModuleConfig.xml을 형제 폴더 fomod_<language>에 기록하거나(기본값, 원본은 그대로 유지됨), 타임스탬프가 찍힌 .bak 복사본을 만든 후 원본 파일 위에 기록합니다. 내보내기는 무손실입니다: 번역된 문자열만 바뀌며, 주석, 인코딩(UTF-8 또는 UTF-16), 알 수 없는 요소, 원본 XML의 서식은 바이트 단위로 보존됩니다. "원래 순서 유지"는 order="Explicit"을 강제하여, 이름순으로 정렬된 목록이 번역된 이름으로 다시 정렬되지 않도록 합니다. 번역된 미리보기는 번역된 복사본에 대해 설치 프로그램 시뮬레이션을 엽니다. 동일한 작업이 명령줄에도 있습니다: ximod-architect translate extract | status | update | apply(§ 5 참조).
번역 패키지 — 세 번째 내보내기 모드는 Nexus에 업로드할 준비가 된 아카이브를 빌드합니다: 기본적으로는 번역된 info.xml과 ModuleConfig.xml(그 원래 경로와 인코딩으로)에 대상 언어로 된 README를 더한 "패치 전용" 패키지로, 사용자가 원본 모드 위에 설치합니다. "모드 전체" 옵션은 그 대신 모드의 모든 파일을 포함합니다(제작자가 재배포를 허용하는지 확인하십시오). 형식(.zip 또는 .7z)과 파일 이름은 이름 템플릿을 따르며, 토큰 {name}, {version}, {LANG}, {lang}, {lang3}, {langname}을 사용합니다(기본값 "{name}_{LANG}", 예: MyMod_FR.zip). 아카이브는 통상적인 진행 창과 함께 백그라운드에서 빌드됩니다.
생산성 — 구성 폴더에 저장되는 언어 쌍별 번역 메모리가 저장하는 모든 번역을 학습합니다. "메모리에서 채우기"는 다음 모드에서 동일한 문자열을 미리 채우고("검토 필요"로 표시됨), 하단 패널은 선택한 문자열에 대해 기억된 번역을 제안합니다. "동일한 문자열에 적용"은 현재 번역을 같은 소스 텍스트를 가진 다른 모든 문자열에 복사합니다. 용어집(도구 행의 토글)은 지정된 용어 — 용어와 그 기대 번역, 선택적으로 대소문자 구분, 또는 번역하지 않고 유지할 용어 — 를 나열하며, 규칙을 어기는 행은 문제로 표시됩니다. CSV 내보내기 / CSV 가져오기는 전체 표(키, 맥락, 소스, 번역, 상태, 메모)를 스프레드시트나 다른 번역자와 교환합니다. 가져오기는 키로 행을 매칭하며 Excel의 구분자를 허용합니다.
고유 텍스트 — 필터 행의 토글은 구별되는 소스 텍스트당 한 행을 나열하며, 그룹의 크기를 알려주는 ×N 배지를 붙입니다. 그 행을 번역하면 같은 텍스트를 가진 모든 문자열이 묻지 않고 자동으로 한 번에 번역됩니다(잠긴 문자열은 제외), 그리고 메시지가 몇 개가 갱신되었는지 알려줍니다. 여러 페이지에서 같은 옵션 이름이나 설명을 반복하는 FOMOD를 가장 빠르게 처리하는 방법입니다. 꺼져 있으면 표는 모든 문자열을 보여 주며, "동일한 문자열에 적용"은 미번역된 것에만 일회성으로 복사하는 데 계속 사용할 수 있습니다.
키보드와 공백 — 표에서 ↑와 ↓는 선택된 행을 이동하고, Page Up과 Page Down은 한 화면 분량의 행만큼 건너뛰며, Home과 End는 첫 행과 마지막 행으로 이동합니다(어떤 번역 필드도 입력 중이 아닐 때. 같은 키가 프로젝트 문자열 창에서도 작동합니다). "공백과 줄 바꿈 표시"는 소스 텍스트의 모든 공백을 표와 상세 창 모두에서 보이게 합니다: 공백은 ·, 줄바꿈 없는 공백은 ␣, 탭은 →, 캐리지 리턴은 ␍, 각 줄 끝에는 ¶.
명령줄에서: ximod-architect translate package <root> <sidecar> [-o OUT] [--format zip|7z] [--full], translate export-csv / import-csv, translate tm-learn / tm-apply; 그리고 batch 매니페스트 항목은 메인 아카이브와 함께 번역 패키지를 생성하기 위해 "translations" 목록(sidecar, mode, out, format, full)을 담을 수 있습니다.

8.14 모드 아카이브 열기

파일 → 아카이브 열기…(또는 .zip/.7z 파일을 창에 끌어다 놓기)는 아카이브를 구성 디렉터리의 작업 폴더(work/<archive name>)로 백그라운드에서 통상적인 진행 창과 함께 추출한 다음, 그 안에서 찾은 FOMOD를 엽니다 — fomod 폴더가 아카이브 최상단에 있든 단일 감싸는 폴더 아래에 있든. 이미 추출된 아카이브는 그대로 재사용되므로, 거기서 한 편집이 유지됩니다. .rar

아카이브는 지원되지 않습니다: 먼저 7-Zip으로 추출하십시오. 번역 창에도 같은 버튼 모드 아카이브 열기…가 있으며, 이것은 fomod 폴더만 추출합니다(translate_work/<archive name>으로). 모드 파일이 거기에 없으므로 "모드 전체" 패키지 옵션은 이 경우 사용할 수 없습니다.
명령줄에서: ximod-architect extract <archive> [-o DIR] [--fomod-only]는 아카이브를 추출하고(기본적으로 그 이름을 딴 폴더로) FOMOD 루트를 출력합니다.

8.15 가져오기 충실도

XIMOD는 이제 과거에 조용히 버렸던 구성을 읽고 다시 기록합니다: 모드 전체 요구 사항(moduleDependencies, 모드 정보 탭의 "모드 요구 사항"에서 다른 곳과 동일한 종속성 편집기로 편집함), 파일과 폴더의 alwaysInstall 및 installIfUsable 속성(파일 표의 "항상" 및 "사용 가능 시" 열), 제목 위치와 색상 및 헤더 이미지 옵션 showImage, showFade, height(모드 정보 → "고급 헤더"), 그리고 작성된 대로 유지되는 단계, 그룹, 옵션의 Ascending/Descending 순서 속성.
아직 모델링할 수 없는 것 — XIMOD이 모르는 스키마 외부 요소 — 은 프로젝트를 로드할 때 보고됩니다: 경고 "이 FOMOD에는 XIMOD가 편집할 수 없는 구조가 N개 포함되어 있습니다. 프로젝트를 저장하면 삭제됩니다"가 문제 패널을 열며, 각 구성과 그 위치(단계, 옵션, 조건부 세트)마다 한 줄씩 표시합니다. 줄을 클릭하면 노드가 선택됩니다. 이 경고는 프로젝트가 열려 있는 한 검증을 거쳐도 문제 패널에 유지됩니다. ximod-architect validate도 같은 경고를 출력합니다.

8.16 백업 및 복구

프로젝트가 기존 버전 위에 저장될 때마다, 이전의 info.xml과 ModuleConfig.xml이 먼저 fomod/backups/<date-time>/로 복사되며(기록하려는 것과 동일하면 아무것도 복사되지 않음), 가장 최근의 N개 폴더만 유지됩니다 — N은 "보관할 백업 수" 설정입니다. 그 폴더는 배포 아카이브나 BA2 패키지에 결코 포함되지 않으며, 그 파일은 결코 고아로 보고되지 않습니다. 두 XML 파일은 원자적으로 기록되므로(임시 파일, 플러시, 이름 변경), 저장 중 충돌이 발생해도 반쯤 기록된 설치 프로그램이 남을 수 없습니다.
파일 → 백업 복원…은 백업을 최신순으로 나열하며, 각각 현재 상태의 프로젝트와의 차이 개수를 함께 보여 줍니다. 비교는 그 백업에 대해 통상적인 비교 창을 열고, 복원은 그것을 되돌릴 수 있는 단일 변경으로 편집기에 로드하며(저장하기 전까지는 디스크에 아무것도 기록되지 않음), 모든 백업 삭제는 — 두 번째 클릭으로 확인됨 — 폴더를 비웁니다. § 8.11의 충돌 보호에 더하여, 이제 수정된 모든 프로젝트의 복구본이 일정한 간격으로 기록됩니다("복구 사본 자동 저장 간격 (분)"). 이것은 정상 종료 시 지워지므로, 복구 프롬프트는 여전히 비정상 종료 후에만 나타납니다.
명령줄에서: ximod-architect backups <root> --list, --restore <timestamp> 또는 --prune N; build와 package도 기록하기 전에 백업을 만듭니다.

8.17 자동 조건으로서의 마스터

플러그인 파일(.esp, .esm 또는 .esl)이 옵션에 추가되면 — 파일 표에서든 끌어다 놓기로든 — XIMOD가 그 헤더를 읽습니다: 프로젝트에 아직 제작자가 없으면 플러그인의 제작자가 채워집니다. 그리고 플러그인이 요구하는 마스터 중 게임 자체의 파일(Categories.json의 "masters"에 게임별로 나열됨)도 아니고 이 모드가 설치하는 것도 아닌 것은, 옵션의 "Active" 파일 조건이 되어 옵션의 기본 유형의 첫 번째 종속성 패턴에 추가됩니다(필요하면 생성됨). 이미 옵션의 파일 조건으로 명명된 마스터는 결코 두 번 추가되지 않습니다. 설정 "플러그인의 마스터를 조건으로 추가"로 이것을 끕니다.
검증은 프로젝트가 설치하는 모든 플러그인(필수 파일, 옵션, 조건부 세트)의 헤더를 읽고, 모드에도 없고 게임의 기본 파일도 아니며 종속성으로 선언되지도 않은 마스터에 대해 경고합니다: "X에는 Y이(가) 필요하지만, 이 모드에 없고 종속성으로도 선언되지 않았습니다". 경고는 노드를 대상으로 하므로, 문제 패널에서 클릭하면 옵션이 선택됩니다.

8.18 Light 플러그인(ESL)과 플러그인 보고서

검증은 light 플러그인 규칙도 확인합니다: light 플래그가 설정되지 않은 .esl 확장자 파일, 새 레코드가 한도를 초과하거나 허용된 FormID 범위를 벗어나는 light로 표시된 플러그인, 그리고 — 정보로서 — light로 표시될 수 있는 .esp(플러그인에 새로운 레코드, 즉 FormID 인덱스가 마스터 수와 같은 레코드를 세어 게임의 한도와 비교합니다: 기본값 2048, Starfield는 4096, Categories.json의 "eslLimit"로 게임별 조정 가능). 64 MB를 초과하는 파일은 세지 않습니다. CLI inspect <plugin>도 같은 수치를 출력합니다.
도구 → 플러그인 보고서…는 이 모든 것을 하나의 표에 모읍니다: 파일, 종류, light 플래그, 마스터(누락된 것은 경고 색으로), 한도 대비 새 레코드, 그리고 적격성. 읽을 수 없는 파일은 그렇게 표시됩니다.

8.19 더 풍부한 시뮬레이터와 시나리오

설치 프로그램 미리 보기(프로젝트 → 설치 프로그램 미리 보기…)의 요약 페이지는 이제 최종 파일 트리로 끝납니다: 게임의 Data 폴더에 놓이게 될 모든 파일, 디스크에서 크기와 함께 펼쳐진 폴더, 그리고 총 설치 크기. 두 항목이 같은 최종 경로에 놓이면 더 높은 우선순위가 이기고(동점은 나중의 선언으로 결정됨) 진 쪽은 취소선과 함께, 마우스를 올리면 이긴 옵션과 함께 표시됩니다. 20,000개 항목을 초과하는 트리는 잘리며 그렇다고 알립니다.
파일 트리는 XIMOD의 모든 트리와 같은 규칙을 따릅니다. 폴더를 오른쪽 클릭하면 펼치기와 접기 하위 메뉴(모두 / 선택 항목 / 선택 항목부터, 마지막 것은 폴더와 그 아래의 모든 것, 설치된 아카이브의 내용을 포함하여 작용함)가 열립니다. 어떤 텍스트 필드에도 포커스가 없을 때 ↑와 ↓는 커서 링을 행에서 행으로 이동하고, Home과 End는 끝으로 이동하며, →는 폴더를 열고(또는 열린 폴더 안으로 들어가고), ←는 폴더를 닫으며(또는 파일에서 폴더로 돌아가고), Enter는 해당 행을 설치하는 구성 요소를 메인 창에서 선택합니다: 옵션, 필수 파일 또는 조건부 세트 — 폴더의 경우 그 첫 번째 파일의 구성 요소.
미리 보기는 또한 모드 관리자처럼 첫 번째 단계 앞의 모드 정보 페이지에서 열립니다: 헤더 이미지(showImage와 height 옵션을 따름), 그 색상의 제목, 제작자, 버전, 웹사이트 링크, 그리고 info.xml의 설명. 다음은 첫 번째 표시 단계로 이끌고, 뒤로는 그리로 돌아가며, 진행 막대는 이를 첫 번째 페이지로 셉니다.
시나리오 행에서는 현재 선택과 파일 가정을 이름으로 저장하고(fomod/scenarios/<name>.json에, 순서 변경에도 견디도록 옵션 이름으로 저장됨), 다시 로드하거나 삭제할 수 있습니다. 선택이 더 이상 어떤 옵션과도 일치하지 않는 시나리오는 몇 개를 찾지 못했는지 보고합니다. 시나리오 폴더는 배포 아카이브에 결코 포함되지 않습니다.
검증은 이제 어떤 옵션도 설정하지 않는 플래그 값을 확인하기 때문에 결코 도달할 수 없는 단계, 옵션, 조건부 세트도 정보로 보고합니다(단계: 이전 단계에 옵션이 없음).
명령줄에서: ximod-architect simulate <root> [--scenario <name|file.json>] [--json]는 표시 단계, 선택, 크기와 덮어쓰기가 있는 최종 설치 목록, 그리고 총계를 출력합니다. batch 매니페스트 항목은 이를 재생하기 위해 "scenarios": [...]를 담을 수 있습니다(시나리오가 더 이상 일치하지 않으면 항목이 실패함).

8.20 옵션별 설치 크기

검증 후(백그라운드에서 파일을 측정함), 각 옵션은 파일 표 위에 그 설치 크기와 파일 수를 표시하며 — 누락된 소스는 경고 색으로 셉니다 — 프로젝트 트리는 옵션 이름 뒤에 크기를 담습니다. 크기는 파일 수정 시간으로 캐시되며 검증할 때마다 갱신됩니다. ximod-architect validate <root> --sizes는 옵션, 필수 파일, 조건부 세트의 총계를 출력합니다.

8.21 Nexus 설명 생성기

도구 → Nexus 설명…은 프로젝트로부터 모드의 Nexus 페이지 텍스트를 빌드합니다: 제목과 설명, 요구 사항(게임과 모드가 제공하지 않는, 설치된 플러그인의 마스터와 선언된 파일 종속성), 설치

옵션(단계당 한 항목, 선택 유형이 있는 그룹, 설명의 첫 문장이 있는 옵션), 설치 참고, 그리고 이전 버전…으로 선택한 FOMOD의 이전 버전에 대해 계산된 변경 내역(추가되거나 제거된 단계, 옵션, 파일, 메타데이터 변경). BBCode(Nexus) 또는 Markdown을 선택하고, 원하는 섹션을 체크하고, 프로젝트에서 발견된 번역 사이드카 중에서 언어를 고른 다음, 복사 또는 다른 이름으로 저장…하십시오. 명령줄에서: ximod-architect nexus-desc <root> [--format bbcode|md] [--previous <root>] [--lang <code>] [-o FILE].

8.22 편집 가능한 조건 편집기

§ 8.7의 조건 편집기는 이제 프로젝트에 작용합니다. 모든 "설정 위치"와 "사용 위치" 줄은 메인 창에서 노드를 선택하는 링크입니다. 각 플래그에는 이름 바꾸기…(인라인 필드; 새 이름이 그 플래그를 설정하는 모든 옵션과 그것을 확인하는 모든 단계 표시, 옵션 패턴, 조건부 세트, 모드 요구 사항에 단일 되돌리기 가능한 변경으로 적용됨; 다른 플래그가 이미 사용 중인 이름은 거부됨)과 모든 사용 삭제가 있습니다. 각 플래그 아래에는 그것이 설정되는 값과 그것이 확인되는 값이 나열됩니다: 확인되지만 결코 설정되지 않는 값은 경고 색으로, 설정되지만 결코 확인되지 않는 값은 회색으로 표시됩니다. 검증은 같은 발견 항목을 보고합니다 — 어떤 옵션도 설정하지 않는 확인된 값에 대해서는 경고(그것을 확인하는 노드를 대상으로 함), 설정되지만 결코 확인되지 않는 플래그에 대해서는 정보.
창 하단의 조건 빌더는 메인 창에서 선택된 노드의 조건 — 단계의 표시 여부, 옵션의 종속성 패턴(패턴 선택, 생성, 유형 변경), 조건부 파일 세트 또는 모드 요구 사항 — 을 편집합니다. 조건을 문장으로 보여 주며(만약 플래그 A = B 그리고 파일 C이(가) Active 그러면 …) 그 위에 조건당 한 행씩 표시합니다: 종류(flag 또는 file), 프로젝트에서 가져온 제안(알려진 플래그와 그 값, 파일 이름, Active/Inactive/Missing)이 있는 이름과 값, 제거 버튼, 그리고 조건 추가. 변경은 실시간으로 적용되고 되돌릴 수 있습니다.
편집기는 이제 독립 창입니다: 메인 창 옆이나 두 번째 화면으로 옮길 수 있고, 한 세션에서 다음 세션으로 위치와 크기를 유지하며, 제목 표시줄 버튼으로 닫힙니다. 플래그 목록은 프로젝트 트리처럼 작동합니다. 플래그 제목을 오른쪽 클릭하면 펼치기(모두 / 선택 항목)와 접기(모두 / 선택 항목) 하위 메뉴가 열립니다. 어떤 텍스트 필드에도 포커스가 없을 때 ↑와 ↓는 커서 링을 한 행에서 다음 행으로 — 플래그 제목과 그 "설정 위치" / "사용 위치" 줄 — 이동하고, →는 닫힌 플래그를 펼치며(또는 열린 것 안으로 들어가고), ←는 열린 플래그를 접고(또는 그 줄 중 하나에서 플래그로 돌아가고), Enter는 그 행을 인스펙터에 보여 줍니다: 줄의 노드, 또는 플래그의 첫 번째 위치. 프로젝트 트리에서 Enter도 같은 일을 합니다: 커서 아래의 행을 선택하면 인스펙터가 따라갑니다.

8.23 프로젝트 문자열 표

도구 → 프로젝트 문자열…은 프로젝트의 모든 이름과 설명(모드 정보, 단계, 그룹, 옵션)을 하나의 표에 나열합니다: 위치, 필드, 텍스트이며, 텍스트는 그 자리에서 편집할 수 있습니다(이름은 한 줄, 설명은 여러 줄). 검색 필드, 종류 필터(모두, 이름, 설명), 중복만 토글이 행을 좁힙니다. 동일한 텍스트(대소문자와 공백 무시)는 ×N으로 배지가 붙고 마우스를 올리면 다른 위치가 표시되어, 복사-붙여넣기된 설명을 찾아내는 데 도움이 됩니다. 찾기/바꾸기 행(대소문자 일치, 전체 단어)은 현재 행이나 모든 표시 행에서 바꿉니다. 위치를 더블 클릭하면 메인 창에서 노드가 선택됩니다.
명령줄에서: ximod-architect strings <root> [--duplicates] [--csv FILE].

8.24 BSA 및 BA2 아카이브 읽기

XIMOD는 이제 베데스다 아카이브를 풀지 않고 그 목차를 읽습니다: BSA 버전 103(Oblivion), 104(Skyrim, Fallout 3/New Vegas), 105(Skyrim Special Edition), 그리고 Fallout 4와 Starfield의 BA2 일반(GNRL) 및 텍스처(DX10) 아카이브(버전 1, 2, 3, 7, 8). Morrowind 아카이브는 명확한 메시지와 함께 거부됩니다. 항목 이름과 크기만 읽히며, 압축된 항목은 그 압축 크기와 함께 나열됩니다.
이것은 세 곳에 공급됩니다. 대상 충돌 검출(§ 8.1)은 이제 옵션이 설치하는 아카이브 내부를 들여다봅니다: 아카이브에 같은 에셋이 들어 있는 두 옵션은 경고로 보고되고("게임의 아카이브

로드 순서가 결정함"), 압축되어 있으면서 동시에 느슨하게 설치되는 에셋은 정보로 보고됩니다("느슨한 파일이 이김"). 시뮬레이터의 최종 파일 트리(§ 8.19)는 각 설치된 아카이브의 내용을 그 아래에 "(아카이브 내)"로 표시하여 나열하며, 총계는 얼마만큼이 아카이브에 압축되어 있는지 보여 줍니다(이중으로 세지 않음). 그리고 도구 → 아카이브 내용…(또는 파일 표에서 아카이브 옆의 눈 버튼)는 항목의 표 — 형식, 개수, 총 압축 해제 크기, 경로, 크기, 압축됨 — 를 검색 필드와 함께 엽니다. 명령줄에서: ximod-architect archive <file.bsa|file.ba2> [--json]; validate도 이제 확실한 대상 충돌을 출력합니다.
실험적 BA2 작성기(CLI ba2)는 이제 새 리더로 그 출력을 다시 읽어 검증됩니다: 경로, 크기, 이름/폴더 해시가 일치합니다.

8.25 중첩 조건 그룹 및 버전 조건

조건은 더 이상 단일 연산자를 가진 하나의 평면 목록에 국한되지 않습니다. 조건을 담는 모든 곳 — 단계의 표시 여부, 옵션의 종속성 패턴, 조건부 파일 세트, 모드 요구 사항 — 은 이제 조건과 중첩 그룹을 담을 수 있는 그룹이며, 각각 자체 And / Or 연산자를 가지고 어떤 깊이로든, FOMOD 스키마가 허용하는 그대로입니다: "(DataVersion = Standard OR DataVersion = Undelayed) AND DataPreset = Aurelia"는 작성된 그대로 유지되어 로드, 편집, 저장됩니다(Aurelia UI와 그 234개의 조건부 세트에서 검증됨). 두 가지 조건 종류도 추가되었습니다: 게임 버전(gameDependency)과 모드 관리자 버전(fommDependency), 각각 최소 버전과 함께.
종속성 편집기에서 그룹 추가는 자체 연산자, 자체 조건, 조건 추가 / 그룹 추가 / 그룹 제거 버튼을 가진 들여쓰기된 틀로 중첩 그룹을 생성합니다. 조건 빌더(§ 8.22)는 중첩 그룹을 그 문장에서 괄호 사이에 보여 주며 동일한 컨트롤로 편집합니다. 시뮬레이터, 검증(설정되지 않고 확인만 되는 플래그, 도달할 수 없는 단계), 충돌 검출, Nexus 설명 모두 중첩 그룹을 이해합니다. 평면 레이아웃으로 저장된 오래된 프로젝트 템플릿도 여전히 로드됩니다.
같은 릴리스의 두 가지 작은 변경: 메뉴 바를 따라 포인터를 미끄러뜨려 연 메뉴는 이제 포인터가 떠나면 닫히고(클릭으로 연 메뉴는 무언가가 클릭될 때까지 유지됨); 그리고 프로젝트 자체의 문자 체계 — 이름과 설명에서의 일본어, 한국어, 중국어, 아랍어, 태국어 등 — 와 편집 중인 번역의 문자 체계를 표시하는 데 필요한 글꼴이 인터페이스 언어와 관계없이 자동으로 로드되어, 번역 창과 Nexus 창의 언어 목록을 포함하여 그 문자 대신 사각형이 나타나는 일이 더 이상 없습니다.
메뉴는 파일, 편집, 프로젝트, 도구, 도움말로 구성되며, 일상적인 작업을 위한 도구 모음이 있고, F1은 이 설명서를 엽니다. 그룹 선택 유형, 옵션 유형, 연산자, 테마는 평이한 언어 레이블("정확히 하나(필수)", "권장", "모든 조건(AND)" 등)과 마우스를 올렸을 때의 설명과 함께 표시되며, 옵션 패널의 각 섹션에는 개념을 설명하는 ⓘ 표시가 있습니다. 확인 대화 상자는 창을 어둡게 하고, Enter와 Esc를 받아들이며, 문제 목록이 길 때는 스크롤됩니다.

8.26 게임 및 해당 카테고리 편집

도구 → 게임 / 카테고리… 는 Categories.json을 직접 편집하지 않고도 모드 정보 탭에 제공되는 게임 목록을 관리할 수 있는 독립된 창을 엽니다. 왼쪽 창에는 게임이 나열되며, 게임 추가, 게임 삭제 및 위로 이동 / 아래로 이동 버튼으로 목록과 순서를 변경합니다. 선택한 게임에 대해 편집기는 식별자(고유한 내부 키), 이름, Nexus slug(링크와 Nexus 설명에 사용), 경량 플러그인 레코드 제한, 기본 마스터 파일(한 줄에 하나씩 — 게임 자체의 masters와 공식 DLC), Nexus 카테고리(한 줄에 하나씩, 표시 순서대로)를 보여줍니다. 입력 항목은 입력하는 동안 검사됩니다. 식별자가 없는 게임, 중복된 식별자 또는 누락된 이름은 표시되며, 모든 문제가 해결될 때까지 저장 은 비활성화된 상태로 유지됩니다. 저장 은 파일을 기록합니다 — 제공된 assets/data/Categories.json이 쓰기 가능하면 해당 파일에, 그렇지 않으면 애플리케이션이 우선 읽는 사용자별 복사본에 기록합니다 — 그리고 새 목록이 즉시 적용됩니다. 닫기 는 확인 후 저장되지 않은 편집을 취소합니다.

8.27 트리 색상

도구 → 트리 색상…은 프로젝트 트리의 각 행 종류 — 모드 정보, 단계, 그룹, 옵션, 필수 파일, 조건부 설치 — 에 고유한 색상을 지정하여 대규모 설치 프로그램의 구조를 한눈에 읽을 수 있게 하며, 시력이 예전 같지 않을 때도 도움이 됩니다(인터페이스 글꼴 크기와 함께 사용하십시오, § 4.5). 창에는 여섯 가지 종류가 색상 견본, 해당 색상의 아이콘, RGB 값과 함께 나열됩니다. "색상 초기화"는 기본값을 복원합니다. 색상 세트는 테마마다 하나씩 있습니다. 창은 현재 사용 중인 테마의 세트를 편집하고 그 사실을 표시하며, 다크와 라이트 테마를 전환하면(§ 4.5) 세트도 바뀝니다. 색상은 Config.ini(부록 A)에 저장됩니다. 선택된 행은 선택 배경 위에서도 읽기 쉽도록 항상 표준 텍스트 색상을 유지합니다.
색상 견본을 클릭하면 XIMOD Translator와 같은 색상 선택기인 "색상 선택" 창이 열립니다. "인덱스 색상" 탭은 AutoCAD 색상 인덱스를 보여 줍니다. 색조와 명암으로 이루어진 24 × 10 격자, 아홉 가지 표준 색상, 회색 단계, 그리고 1부터 255까지의 인덱스를 입력하는 "색상:" 필드입니다. "트루 컬러" 탭은 색조 / 채도 영역, 휘도 슬라이더, H, S, L 필드, 현재 색상의 빨강, 초록, 파랑 값, 그리고 255,128,0과 같은 값을 입력할 수 있는 "RGB 색상:" 필드를 제공합니다. "확인"은 색상을 즉시 적용하고 — 창 뒤에서 트리가 다시 그려집니다 — 저장합니다. "취소" 또는 Esc는 이전 색상을 유지합니다.

8.28 마지막 위치를 기억하는 파일 대화 상자

옵션에 소스를 추가하거나("파일 추가", "폴더 추가") 이미지를 찾아볼 때, 대화 상자는 더 이상 매번 프로젝트 루트에서 시작하지 않고 마지막으로 선택한 파일이나 폴더의 상위 폴더에서 열립니다. 한 옵션에 Feature D\A를 선택하면 다음 대화 상자는 Feature D에서 열리고 B와 C가 바로 눈앞에 있습니다. 폴더 안의 파일을 선택하면 다음 대화 상자는 그 폴더에서 열립니다. 이 기억은 세션 동안 유지되며 프로젝트가 바뀌면 초기화됩니다. 기억된 폴더가 루트 밖에 있거나 사라졌다면 대화 상자는 루트로 돌아갑니다. 대규모 설치 프로그램에서는 여전히 탐색기가 가장 빠른 방법입니다. 기능 폴더를 한 번 연 다음 각 하위 폴더를 해당 옵션에 끌어다 놓으십시오(§ 8.5).

8.29 프리셋 마법사

프로젝트 → 프리셋 마법사…는 현재 프로젝트에 § 7.5에서 설명한 "프리셋 + 사용자 지정" 패턴을 구축합니다. 사용자는 프리셋을 고르고 끝내거나, 프리셋을 고른 뒤에도 몇 가지 기능을 조정하거나, 모든 것을 직접 구성할 수 있으며 — 어떤 파일도 두 번 설치되는 일은 없습니다. 이를 직접 하려면 모든 옵션의 플래그, 동적 유형, 표시 조건, 그리고 옵션마다 하나의 조건부 설치가 필요합니다. 마법사는 이 모든 것을 실행 취소 가능한 한 번의 작업으로 처리합니다.
“기존 페이지에서” 탭에서 기능 페이지인 단계를 선택하고(프리셋은 그 옵션 중에서 고릅니다), 생성될 이름 — 설치 모드 단계, 프리셋 그룹, “프리셋 따르기” 및 “사용자 지정” 옵션, 프리셋 플래그, “없음” 및 “프리셋 없음” 옵션의 이름 — 을 확인한 다음 프리셋을 정의하십시오. “프리셋

추가"는 열을 만들고, 행렬은 선택한 모든 페이지의 모든 옵션을 각각 하나의 행으로 보여 줍니다. 배타적 그룹(SelectExactlyOne / SelectAtMostOne)에서는 프리셋마다 옵션 하나 또는 "(없음)"을 고르고, 다중 선택 그룹에서는 원하는 만큼 선택합니다. 아래쪽 막대는 첫 번째 문제(선택된 페이지 없음, 프리셋 없음, 이름 누락, 이름이 같은 두 프리셋)를 보고하며, 문제가 해결될 때까지 "적용"은 비활성화된 상태로 유지됩니다.

"적용"은 설치 모드 단계를 삽입합니다(맨 앞에, 또는 이전 실행이 남겨 둔 위치에). 이 단계에는 프리셋들과 "프리셋 없음(직접 구성)" 중 하나를 고르는 단일 선택인 "프리셋" 그룹이 있으며, 각 옵션은 preset 플래그만 기록합니다. 그다음 기능 페이지마다 두 옵션으로 이루어진 그룹 — "프리셋의 선택 따르기" 또는 "사용자 지정" — 이 하나씩 오며, 이 그룹만이 해당 페이지의 "사용자 지정" 플래그를 기록합니다. 각 기능 페이지는 해당 "사용자 지정" 옵션이 선택되었을 때만 표시됩니다. 마법사는 그 옵션에 플래그를 추가하고, 프리셋이 고른 변형을 그 프리셋 아래에서 Recommended로 표시하며, 배타적 그룹에 "없음" 옵션을 추가하고(기본적으로 권장, 무언가를 고르는 프리셋 아래에서는 선택 사항. SelectAtMostOne 그룹은 SelectExactlyOne이 되므로 모드 관리자가 두 번째 "없음"을 표시하지 않습니다), 옵션의 파일을 "조건부 설치 항목" 탭으로 옮깁니다. 옵션마다 하나의 세트이며, 페이지가 사용자 지정되었고 거기서 옵션이 선택된 경우, 또는 페이지가 프리셋에 맡겨졌고 그 프리셋이 해당 옵션을 고른 경우에 설치됩니다. 어떤 플래그도 함께 선택할 수 있는 두 옵션이 기록하는 일은 없으므로, 설치 프로그램에서 뒤로 돌아가 무엇을 변경하든 항상 사용자가 최종적으로 보는 것이 설치됩니다(§ 7.5). 메시지가 수행된 작업을 요약하며, 편집 → 실행 취소로 전체 작업을 되돌릴 수 있습니다. 그런 다음 프로젝트 → 설치 프로그램 미리 보기…(프리셋만, 프리셋과 사용자 지정한 페이지, 페이지만)와 이 패턴을 인식하는 프로젝트 → FOMOD 유효성 검사(§ 8.1)로 결과를 확인하십시오.

"새 스켈레톤" 탭은 아직 페이지가 없는 프로젝트를 위한 것입니다. 각 기능을 페이지 이름, 선택 종류(배타적 또는 다중), 한 줄에 하나씩의 변형으로 기술하면, "페이지 만들기"가 이를 — 파일 없이 — 프로젝트에 추가하고 해당 페이지가 선택된 상태로 첫 번째 탭으로 전환합니다. 그곳에서 프리셋을 정의하고 적용한 다음 파일을 지정하십시오. 직접 고르는 것은 페이지에, 프리셋이 설치하는 것은 마법사가 만든 조건부 설치에 지정합니다.

마법사가 이미 변환한 프로젝트에서 마법사를 다시 실행해도 첫 번째 구조 위에 두 번째 구조가 쌓이지 않습니다. 창은 기존 페이지, 이름, 프리셋을 불러온 상태로 열리며 그 사실을 알려 줍니다. 프리셋을 추가하거나 변경하고 필요하면 페이지를 더 선택한 다음 "적용"을 누르면, 실행 취소할 수 있는 한 번의 작업으로 전체 구조를 다시 만들고 플래그 이름을 유지하므로 모드의 업데이트가 이전 릴리스와 계속 호환됩니다. 버전 2.1.0 또는 2.2.1로 만든 구조도 인식되어 현재 방식으로 다시 만들어집니다. 마법사를 다시 실행하고 적용한 다음 업데이트를 게시하십시오.

8.30 FOMOD 간 잘라내기, 복사, 붙여넣기

단계, 그룹, 옵션은 잘라내고 복사하고 붙여넣을 수 있습니다. 편집 → 잘라내기, 복사, 붙여넣기, 트리 노드의 오른쪽 클릭 메뉴에 있는 같은 명령, 또는 텍스트 필드에 포커스가 없는 동안의 Ctrl+X, Ctrl+C, Ctrl+V를 사용합니다 — 텍스트 필드 안에서는 이 키들이 계속 텍스트에 작용합니다. 복사는 노드를 그 안에 포함된 모든 것과 함께 가져갑니다: 그룹과 옵션, 파일, 이미지, 플래그, 조건, 종속성 패턴. 잘라내기는 노드를 프로젝트에서 제거하고 클립보드에 보관하며, 편집 → 실행 취소로 원래대로 되돌릴 수 있습니다.

붙여넣기는 선택을 따릅니다: 단계는 선택한 단계 뒤에 삽입되고(아무것도 선택되지 않았으면 맨 끝에), 그룹은 선택한 단계 안의, 선택한 그룹이 있으면 그 뒤에, 옵션은 선택한 그룹 안의, 선택한 옵션 뒤에 들어갑니다. 붙여넣은 노드가 선택되며, 붙여넣기는 실행 취소 가능한 하나의 변경입니다. 선택 항목이 노드를 받을 수 없을 때 — 단계가 선택되지 않은 상태의 그룹, 그룹이 없는 상태의 옵션 — 무엇을 선택해야 하는지 메시지가 알려 줍니다.

클립보드는 프로젝트가 아니라 애플리케이션에 속합니다: 한 FOMOD 탭에서 복사하고, 다른 탭으로 전환한 뒤 붙여넣으십시오. 복사본은 시스템 클립보드에도 저장되므로, 한 XIMOD 창에서 복사한 노드를 다른 XIMOD 창에 붙여넣을 수 있습니다. 경로는 모드 폴더 기준 상대 경로이므로, 다른 모드에서 가져온 노드는 이 모드에 없는 파일을 가리킬 수 있습니다: 붙여넣은 뒤 XIMOD는 그 노드의 파일 중 몇 개가 이 모드 폴더에 없는지 알려 주고(이후 프로젝트 → 누락된 파일 찾기…,

§ 8.32가 경로를 다시 지정하도록 도와줍니다), 노드가 검사하지만 이 프로젝트의 어떤 옵션도 설정하지 않는 플래그를 나열합니다 — 결코 나타나지 않을 페이지나 유형이 결코 바뀌지 않을 옵션을 즉시 잡아냅니다.

8.31 디스크에서 소스 이름 바꾸기

파일 탐색기에서 파일이나 폴더의 이름을 바꾸면 프로젝트가 그것에 대해 가진 모든 참조가 깨집니다. 파일 표 — 옵션의 소스 표(§ 4.2), 필수 설치 항목 탭(§ 4.3), 조건부 세트의 파일(§ 4.4) — 에 있는 "디스크에서 이름 바꾸기…" 버튼은 대신 선택한 파일이나 폴더의 이름을 XIMOD에서 바꾸고, 동시에 참조도 업데이트합니다: 그것을 사용하는 모든 옵션, 필수 파일, 조건부 세트, 그리고 폴더의 경우 그 안에 있는 항목에 대한 모든 참조. 이 버튼은 프로젝트에 루트 디렉터리가 생기면 사용할 수 있습니다.

창에는 현재 경로, 새 이름을 입력하는 필드(항목은 같은 폴더에 남습니다), 함께 업데이트될 참조의 수가 표시됩니다. "이름 바꾸기" 또는 Enter로 적용합니다. Windows에서 금지하는 문자(: * ? 또는 | 등)를 포함하거나 마침표나 공백으로 끝나는 이름, 해당 폴더에서 이미 사용 중인 이름, 더 이상 존재하지 않는 소스는 창 안에 보고되며, 창은 열린 채로 유지됩니다. 대소문자만 바꾸는 변경(main.esp → Main.esp)은 모든 시스템에서 작동하며, 각 참조는 자신의 구분 기호 스타일(\ 또는 /)을 유지합니다. 항목의 이름은 디스크에서 즉시 바뀝니다: 편집 → 실행 취소는 프로젝트의 참조를 복원할 뿐, 디스크상의 이름은 복원하지 않습니다.

8.32 누락된 파일 찾기

파일은 XIMOD 밖에서도 이름이 바뀌고 이동됩니다 — 모드를 재구성하는 동안이나 모드의 새 버전을 압축 해제한 뒤에. 프로젝트 → 누락된 파일 찾기…는 모드 폴더를 검색하여 프로젝트가 사용하지만 더 이상 존재하지 않는 모든 파일, 폴더, 이미지를 경로마다 한 번씩, 그것을 사용하는 참조의 수와 함께 나열합니다.

각 항목에 대해 창은 모드 폴더에서 찾은 후보를 제안합니다: 다른 폴더에 있는 같은 이름(이동됨), 또는 원래 있던 폴더에서 프로젝트가 아직 사용하지 않는 비슷한 이름으로, 파일인 경우 확장자가 같은 것(이름 변경됨). 같은 이름의 항목이 다른 곳에 정확히 하나 있으면 그것이 미리 선택됩니다. 목록에서 다른 후보를 고르거나, (그대로 두기)를 선택해 그대로 유지하거나, "찾아보기…"를 클릭하여 모드 폴더의 아무 파일이나 폴더를 지정하십시오.

"적용"은 선택한 모든 참조를 한 번에 — 폴더의 경우 그 안의 참조까지 — 실행 취소 가능한 하나의 변경으로 업데이트한 다음, 모드 폴더를 다시 검색합니다. "다시 검색"은 직접 파일을 옮긴 뒤에 같은 작업을 수행합니다. 파일 감시 기능과 달리 이 기능은 XIMOD가 닫혀 있는 동안 바뀐 것도 잡아내며, 사용자의 확인 없이 프로젝트를 변경하는 일은 절대 없습니다. 여전히 누락된 파일은 프로젝트 → FOMOD 유효성 검사(§ 4.8)가 계속 보고합니다.

부록

부록 A — Config.ini 파일

구성은 실행 파일 옆에 위치한 Config.ini 파일에 저장됩니다. 완전한 예시는 다음과 같습니다:

```
; XIMOD Architect Configuration
[General]
Locale=fra
Country=FRA
FirstStart=1
Theme=Dark
FontSize=14
ReplaceNewlines=1
MaxRecentFiles=10
SplashScreenSeconds=2

[Window]
WindowWidth=1280
WindowHeight=800

[WindowPositions]
WinPos_ximod_translation=120,80
WinSize_ximod_translation=1100,700
WinPos_ximod_preview=200,140
WinSize_ximod_preview=900,650

[Scripts]
PreSaveScript=
PostSaveScript=

[RecentFiles]
RecentFile0=C:\Mods\MyMod
```

이전 것들에 두 개의 키가 추가됩니다: Country(선택한 국가의 ISO 3166-1 alpha-3 코드)와 FirstStart(§ 2.5 참조).

[WindowPositions] 섹션은 자유 창의 기하 구조를 기억합니다(§ 2.6 참조). 각 창에 대해 WinPos_<id>는 그 위치(x,y)를, WinSize_<id>는 그 크기(width,height)를 저장합니다. 이 항목들은 사용자가 실제로 창을 이동하거나 크기를 조정한 후에만 생성됩니다. 그것들이 없으면 창은 메인 창 중앙에 열립니다. 메인 창의 크기는 [Window](WindowWidth / WindowHeight)에 보관되지만, 그 위치는 보관되지 않습니다.

부록 B — 키보드 단축키

macOS에서 Ctrl 키는 Cmd에 해당합니다.

전역 단축키

단축키	동작
Ctrl+N	새 프로젝트.
Ctrl+O	폴더 열기.
Ctrl+Shift+O	파일 열기.
Ctrl+S	저장(FOMOD 생성).
Ctrl+,	설정 열기.
Ctrl+Q	애플리케이션 종료.
F1	사용자 설명서 열기(도움말 → 사용자 설명서).
Ctrl+X / Ctrl+C / Ctrl+V	트리에서 선택한 단계, 그룹 또는 옵션을 잘라내거나 복사하거나 붙여넣습니다(§ 8.30). 텍스트 필드에서는 이 키가 텍스트에 작용합니다.

창 및 대화 상자

단축키	동작
Esc	활성 자유 창(국가 선택, 속성, XML 편집기) 또는 모달 대화 상자(설정, 정보, 확인, 검증 보고서, 스크립트)를 닫습니다. 확인 대화 상자에서는 "취소"로 작동합니다. 번역 편집기나 메인 창에는 효과가 없습니다.

목록 탐색 — 속성 및 번역 편집기

키	동작
↑ / ↓	선택을 한 행 위 / 아래로 이동합니다.
Page Up / Page Down	한 화면씩 스크롤합니다. 그 단계는 창 높이에 맞게 조정됩니다.
Home / End	첫 / 마지막 행으로 이동합니다(마지막 행이 완전히 보이도록 표시됩니다).
Enter	속성: 강조된 항목을 선택합니다. 번역 편집기: 행의 입력 필드에 커서를 둡니다.
Tab / Shift+Tab	속성: 다음 / 이전 행(플래그에는 놓이지 않음). 번역 편집기(편집 중): 다음 / 이전 번역 필드.

플래그 격자 — "국가 선택" 창

키	동작
↑ ↓ ← → (또는 8 2 4 6)	격자 내에서 선택 틀을 이동합니다.
Page Up / Page Down	한 화면의 플래그를 스크롤합니다.
Home / End	첫 / 마지막 플래그로 이동합니다(마지막 행이 완전히 보이도록 표시됩니다).
Tab / Shift+Tab	다음 / 이전 플래그.
Enter	강조된 국가를 확정합니다.
Esc	창을 닫습니다.

부록 C — 용어집

용어	정의
FOMOD	info.xml과 ModuleConfig.xml을 기반으로 하는 모드 설치 프로그램 형식. 베데스다 게임의 표준이며, 대부분의 다른 게임에서도 Vortex와 Mod Organizer 2가 지원합니다.
Step	하나 이상의 그룹을 포함하는 설치 마법사의 페이지.
Group	선택 유형을 공유하는 옵션(플러그인)의 집합.
Plugin	설치할 파일과 연결된, 사용자에게 제공되는 옵션.
Flag	설치를 제어하는 데 사용되는 조건 플래그(이름 = 값 쌍).
Endonym	그 언어로 쓰인 국가나 언어의 이름.
ISO 639-3	번역 폴더의 이름을 짓는 세 글자 언어 코드(예: fra, eng).
ISO 3166-1 alpha-3	세 글자 국가 코드(예: FRA, USA).

용어	정의
루트 디렉터리	모드 파일과 생성된 fomod 하위 폴더를 포함하는 작업 폴더.

부록 D — 완전한 FOMOD 예제

이 부록은 가상이지만 완전한 프로젝트 — Skyrim Special Edition용 조명 모드 "Dawnlight" — 를 제시하며, XIMOD Architect의 모든 가능성을 실습합니다: 필수 파일, 여러 단계, 단계 표시 조건, 다섯 가지 그룹 유형, 다섯 가지 옵션 유형, 조건 플래그, 동적 옵션 유형(종속성 패턴), 조건부 설치. 아래 두 파일은 하나의 프로젝트를 이루며, XIMOD이 생성하는 그대로 재현되었습니다(들여쓰기 단순화).

D.1 info.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<fomod>
    <Name>Dawnlight</Name>
    <Author>Patrice</Author>
    <Version>1.2.0</Version>
    <Website>https://www.nexusmods.com/skyrimspecialedition/mods/98765</Website>
    <Description>Improved lighting, candles and weather for Skyrim SE.</Description>
    <Groups>Visuals and Graphics</Groups>
    <Game>skyrimSpecialEdition</Game>
</fomod>
```

D.2 ModuleConfig.xml

예제가 다루는 기능:

- 필수 파일: requiredInstallFiles(파일 하나와 폴더 하나).
- 단계 1: SelectAll 그룹(Required 옵션)과 SelectExactlyOne(Recommended / Optional 옵션, res 플래그).
- 단계 2: SelectAny 그룹(candles 및 moon 플래그)과 SelectAtMostOne.
- 단계 3: 표시 조건(<visible>) 다음에, 동적 유형 옵션(dependencyType: candles=On이면 Recommended, 파일이 없으면 NotUsable)과 CouldBeUsable 옵션을 가진 SelectAtLeastOne 그룹.
- 조건부 설치: 두 개의 패턴(And 및 Or 연산자, 플래그 및 파일 종속성, Active / Inactive / Missing 상태).

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>Dawnlight</moduleName>
  <moduleImage path="fomod\header.png"/>
  <requiredInstallFiles>
    <file source="Dawnlight.esp" destination="Dawnlight.esp" priority="0"/>
    <folder source="core\meshes" destination="meshes" priority="0"/>
  </requiredInstallFiles>
  <installSteps order="Explicit">
    <installStep name="Main installation">
      <optionalFileGroups>
        <group name="Core module" type="SelectAll">
          <plugins order="Explicit">
            <plugin name="Base files">
              <description>Scripts and meshes essential to the mod.</description>
              <files>
                <folder source="core\scripts" destination="scripts" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Required"/>
              </typeDescriptor>
            </plugin>
          </plugins>
        </group>
        <group name="Texture resolution" type="SelectExactlyOne">
```

```
        <plugins order="Explicit">
          <plugin name="2K">
            <description>2K textures, good quality/performance balance.</description>
            <image path="fomod\2k.png"/>
            <conditionFlags>
              <flag name="res">2K</flag>
            </conditionFlags>
            <files>
              <folder source="tex\2k" destination="textures" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Recommended"/>
            </typeDescriptor>
          </plugin>
          <plugin name="4K">
            <description>4K textures, for powerful setups.</description>
            <image path="fomod\4k.png"/>
            <conditionFlags>
              <flag name="res">4K</flag>
            </conditionFlags>
            <files>
              <folder source="tex\4k" destination="textures" priority="1"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
  <installStep name="Ambience options">
    <optionalFileGroups>
      <group name="Lighting effects" type="SelectAny">
        <plugins order="Explicit">
          <plugin name="Dynamic candles">
            <description>Adds candles with flickering light.</description>
            <conditionFlags>
              <flag name="candles">On</flag>
            </conditionFlags>
            <files>
              <folder source="opt\candles" destination="meshes" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
          <plugin name="Enhanced moon">
            <description>Increases the intensity of the moonlight.</description>
            <conditionFlags>
              <flag name="moon">On</flag>
            </conditionFlags>
            <files>
              <folder source="opt\moon" destination="textures" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
      <group name="Sound ambience" type="SelectAtMostOne">
        <plugins order="Explicit">
          <plugin name="Night sounds">
            <description>Adds a nighttime sound ambience.</description>
            <files>
              <folder source="opt\sound" destination="sound" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
  <installStep name="ENB compatibility">
```

```
    <visible>
      <dependencies operator="And">
        <flagDependency flag="res" value="4K"/>
        <fileDependency file="KnownConflict.esp" state="Inactive"/>
      </dependencies>
    </visible>
    <optionalFileGroups>
      <group name="ENB preset" type="SelectAtLeastOne">
        <plugins order="Explicit">
          <plugin name="Light ENB">
            <description>Low-cost ENB preset.</description>
            <files>
              <file source="enb\light.ini" destination="enblocal.ini" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
          <plugin name="Ultra ENB">
            <description>High-end ENB preset (dynamic type).</description>
            <files>
              <file source="enb\ultra.ini" destination="enblocal.ini" priority="1"/>
            </files>
            <typeDescriptor>
              <dependencyType>
                <defaultType name="Optional"/>
                <patterns>
                  <pattern>
                    <dependencies operator="And">
                      <flagDependency flag="candles" value="On"/>
                    </dependencies>
                    <type name="Recommended"/>
                  </pattern>
                  <pattern>
                    <dependencies operator="Or">
                      <fileDependency file="EnbHelper.dll" state="Missing"/>
                    </dependencies>
                    <type name="NotUsable"/>
                  </pattern>
                </patterns>
              </dependencyType>
            </typeDescriptor>
          </plugin>
          <plugin name="Experimental extension">
            <description>Additional effects, not guaranteed.</description>
            <files>
              <folder source="enb\extra" destination="textures" priority="2"/>
            </files>
            <typeDescriptor>
              <type name="CouldBeUsable"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
</installSteps>
<conditionalFileInstalls>
  <patterns>
    <pattern>
      <dependencies operator="And">
        <flagDependency flag="res" value="4K"/>
        <flagDependency flag="candles" value="On"/>
      </dependencies>
      <files>
        <folder source="cond\candles4k" destination="textures" priority="0"/>
      </files>
    </pattern>
    <pattern>
      <dependencies operator="Or">
        <flagDependency flag="moon" value="On"/>
        <fileDependency file="SkyUI_SE.esp" state="Active"/>
      </dependencies>
      <files>
        <file source="cond\patch.esp" destination="patch.esp" priority="0"/>
      </files>
    </pattern>
```

```
    </patterns>
  </conditionalFileInstalls>
</config>
```

부록 E — 라이선스 및 출처 표기

XIMOD Architect의 소스 코드는 MIT 라이선스(© 2025-2026)로 배포됩니다. 아래의 제3자 구성 요소는 각자의 라이선스를 유지합니다. 전체 세부 사항과 출처 표기는 애플리케이션과 함께 제공되는 THIRD-PARTY-NOTICES 파일에 나옵니다.

구성 요소	출처	라이선스
XIMOD Architect 코드	—	MIT
Noto 글꼴	Google Fonts / Noto 프로젝트	SIL Open Font License 1.1
플래그(SVG)	drapeauxdespays.fr	로열티 프리(상업적 사용 포함)
ISO 3166-1 alpha-3 국가 코드	ISO — iso.org/obp	자유 접근
ISO 639-3 언어 코드	SIL — iso639-3.sil.org	출처 표기 필요(아래 참조)
Rust 라이브러리	crates.io 생태계	MIT / Apache-2.0(대부분)

ISO 639-3 출처 표기: 언어 코드는 iso639-3.sil.org에서 가져온 것입니다(출처를 표기해야 함). XIMOD이 추가한 데이터 — 엔도님, 연결된 글꼴, 국가 ↔ 언어 매핑 — 는 ISO 639-3 표준의 일부가 아니며, 표준의 식별자는 수정되지 않습니다. 더욱이 애플리케이션은 전체 코드 집합을 재배포할 수 있는 어떠한 수단도 제공해서는 안 됩니다.